BBM 201 – DATA STRUCTURES

HACETTEPE UNIVERSITY

DEPT. OF COMPUTER ENGİNEERİNG

RED-BLACK TREES

Acknowledgement: The course slides are adapted from the slides prepared by R. Sedgewick and K. Wayne of Princeton University.

Text

implementation	worst-case cost (after N inserts)			average case (after N random inserts)			ordered iteration?
	search	insert	delete	search hit	insert	delete	
sequential search (unordered list)	N	N	N	N/2	N	N/2	no
binary search (ordered array)	lg N	N	N	lg N	N/2	N/2	yes
BST	N	N	N	1.39 lg N	1.39 lg N	?	yes
goal	log N	log N	log N	log N	log N	log N	yes

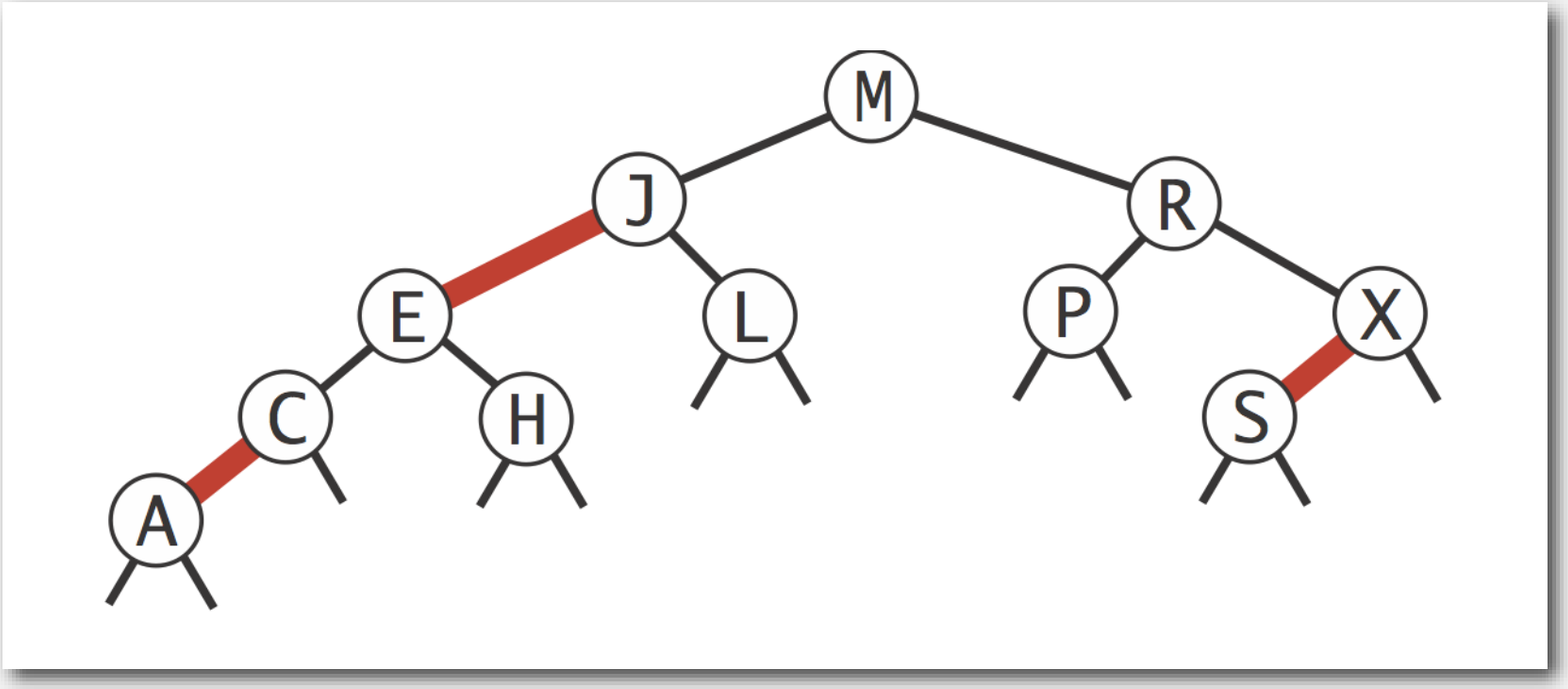
- Challenge. Guarantee performance.

An equivalent definition

A BST such that:

- No node has two red links connected to it.
- Every path from root to null link has the same number of black links.
- Red links lean left (correct ordering)

"perfect black balance"

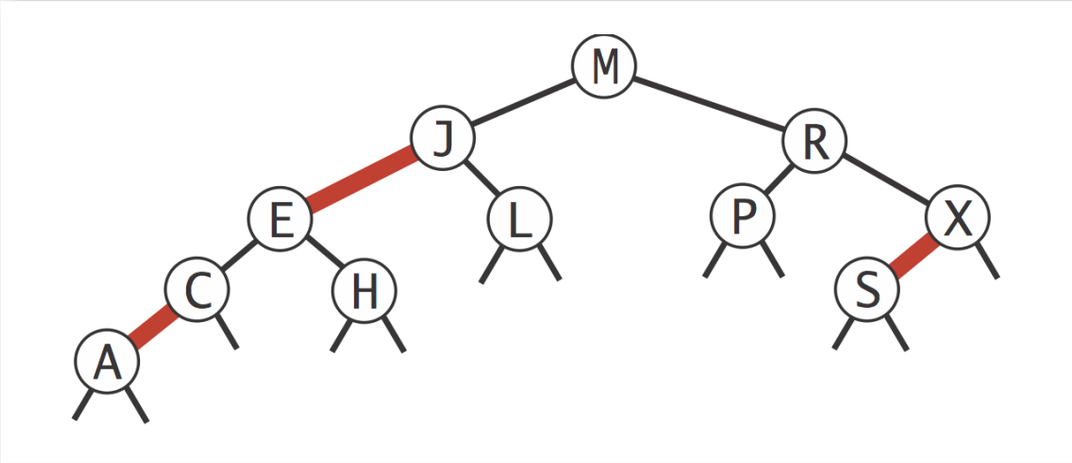


Search implementation for red-black BSTs

Observation. Search is the same as for elementary BST (ignore color).

but runs faster because of better balance

```
public Val get(Key key)
{
   Node x = root;
   while (x != null)
   {
      int cmp = key.compareTo(x.key);
      if      (cmp  < 0) x = x.left;
      else if (cmp  > 0) x = x.right;
      else if (cmp == 0) return x.val;
   }
   return null;
}
```

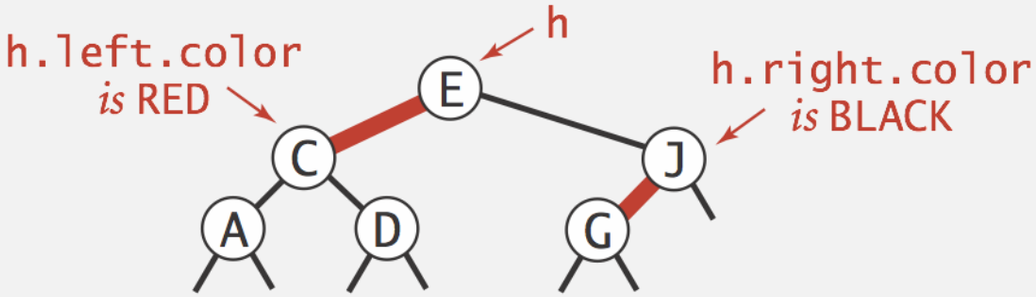


Red-black BST representation

Each node is pointed to by precisely one link (from its parent) $\Rightarrow$ can encode color of links in nodes.

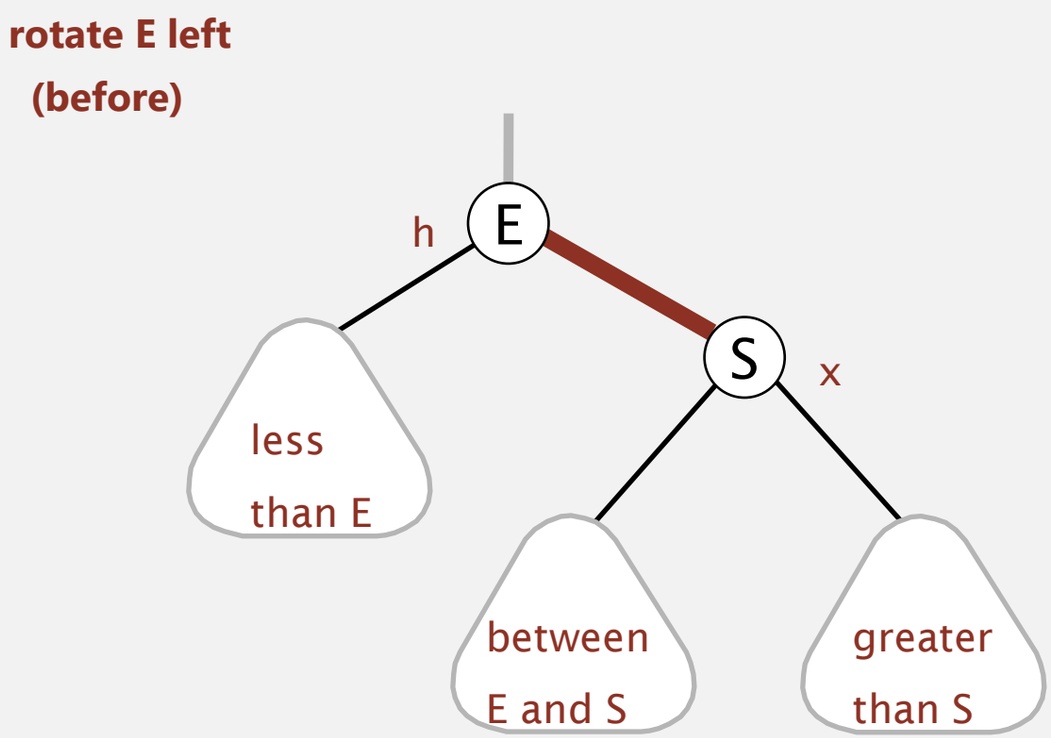
```
private static const boolean RED   = true;
private static const boolean BLACK = false;

private class Node
{
   Key key;
   Value val;
   Node left, right;
   bool color;    // color of parent link
}

private bool isRed(Node x)
{
   if (x == null) return false;
   return x.color == RED;
}
```

null links are black



Elementary red-black BST operations

Left rotation. Orient a (temporarily) right-leaning red link to lean left.

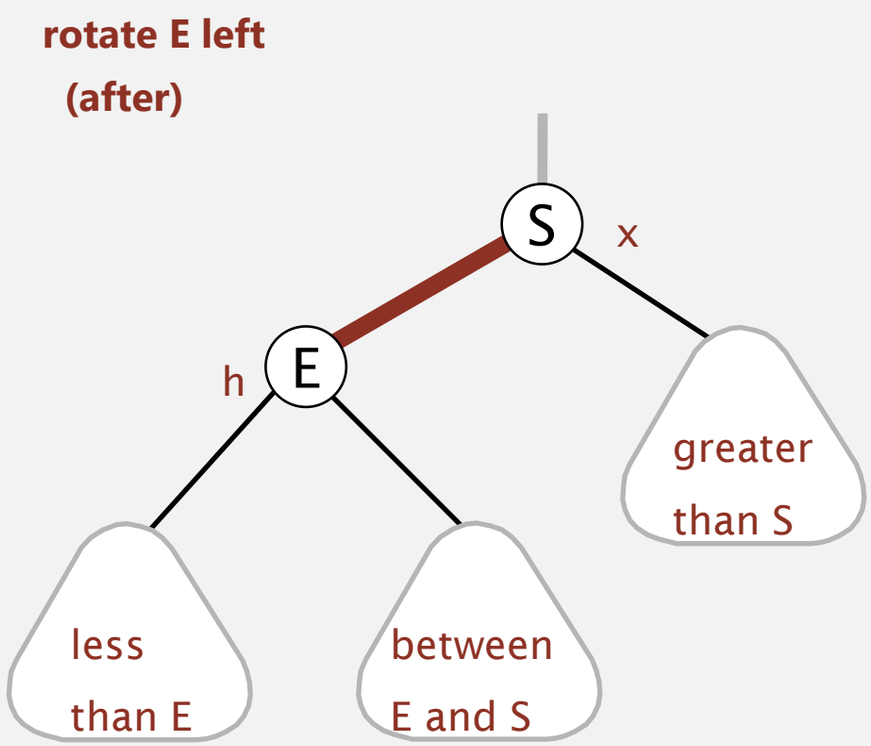



```
private Node rotateLeft(Node h)
{
   assert isRed(h.right);
   Node x = h.right;
   h.right = x.left;
   x.left = h;
   x.color = h.color;
   h.color = RED;
   return x;
}
```

Invariants. Maintains symmetric order and perfect black balance.

Elementary red-black BST operations

Left rotation. Orient a (temporarily) right-leaning red link to lean left.

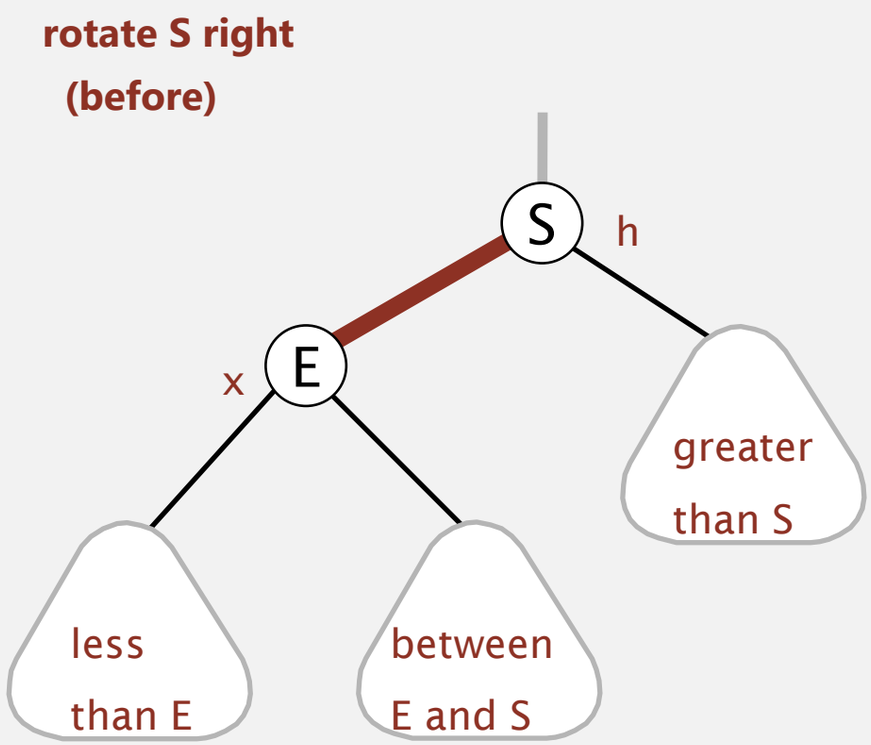



```
private Node rotateLeft(Node h)
{
   assert isRed(h.right);
   Node x = h.right;
   h.right = x.left;
   x.left = h;
   x.color = h.color;
   h.color = RED;
   return x;
}
```

Invariants. Maintains symmetric order and perfect black balance.

Elementary red-black BST operations

Right rotation. Orient a left-leaning red link to (temporarily) lean right.

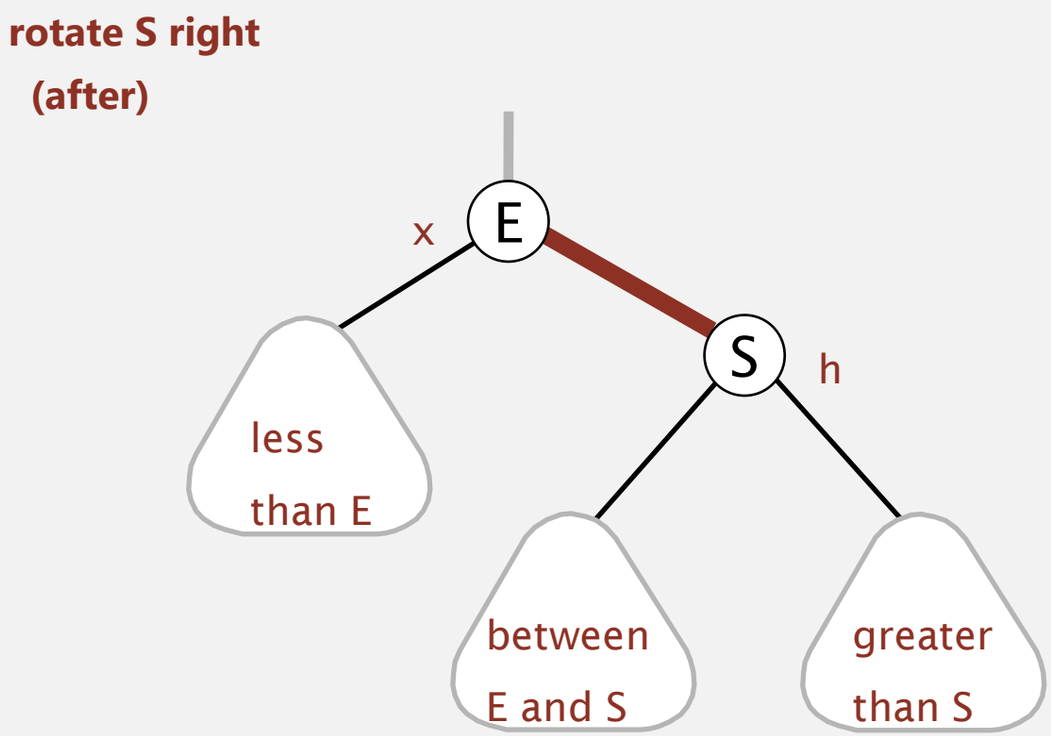



```
private Node rotateRight(Node h)
{
   assert isRed(h.left);
   Node x = h.left;
   h.left = x.right;
   x.right = h;
   x.color = h.color;
   h.color = RED;
   return x;
}
```

Invariants. Maintains symmetric order and perfect black balance.

Elementary red-black BST operations

Right rotation. Orient a left-leaning red link to (temporarily) lean right.

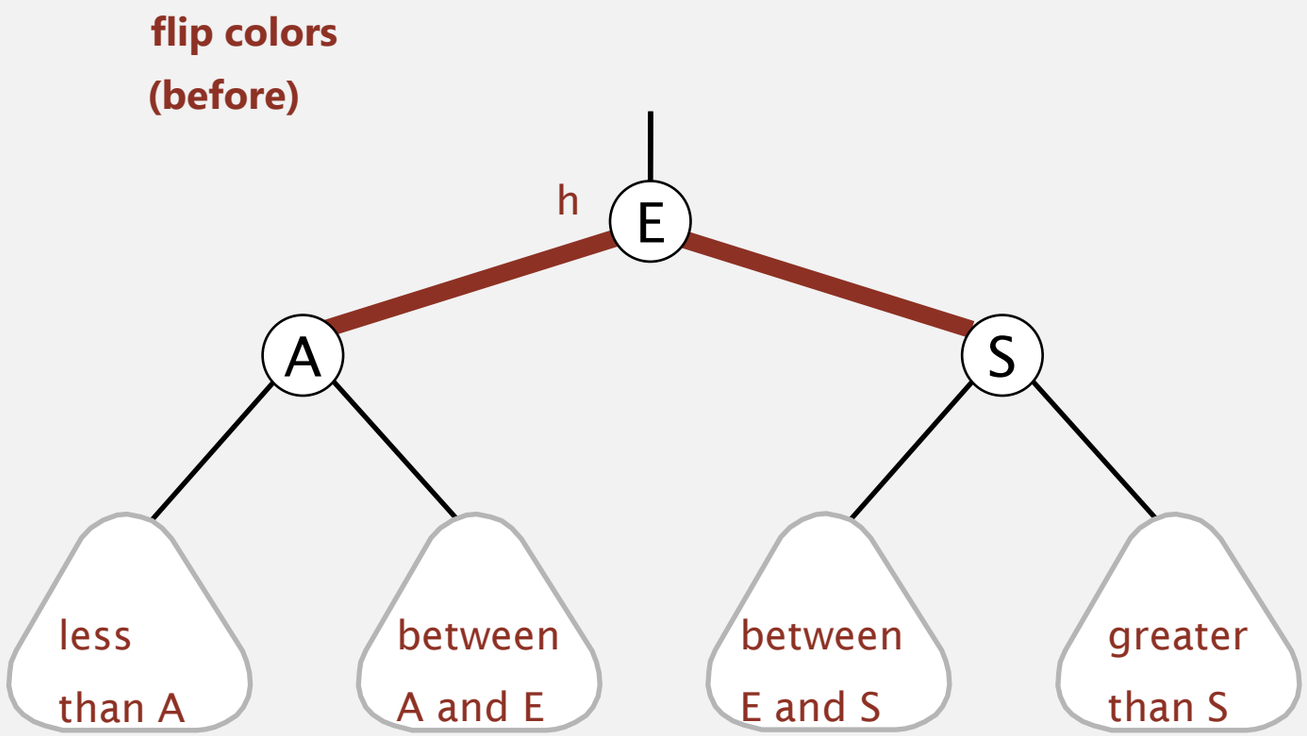



```
private Node rotateRight(Node h)
{
   assert isRed(h.left);
   Node x = h.left;
   h.left = x.right;
   x.right = h;
   x.color = h.color;
   h.color = RED;
   return x;
}
```

Invariants. Maintains symmetric order and perfect black balance.

Elementary red-black BST operations

Color flip. Recolor to split a (temporary) 4-node.

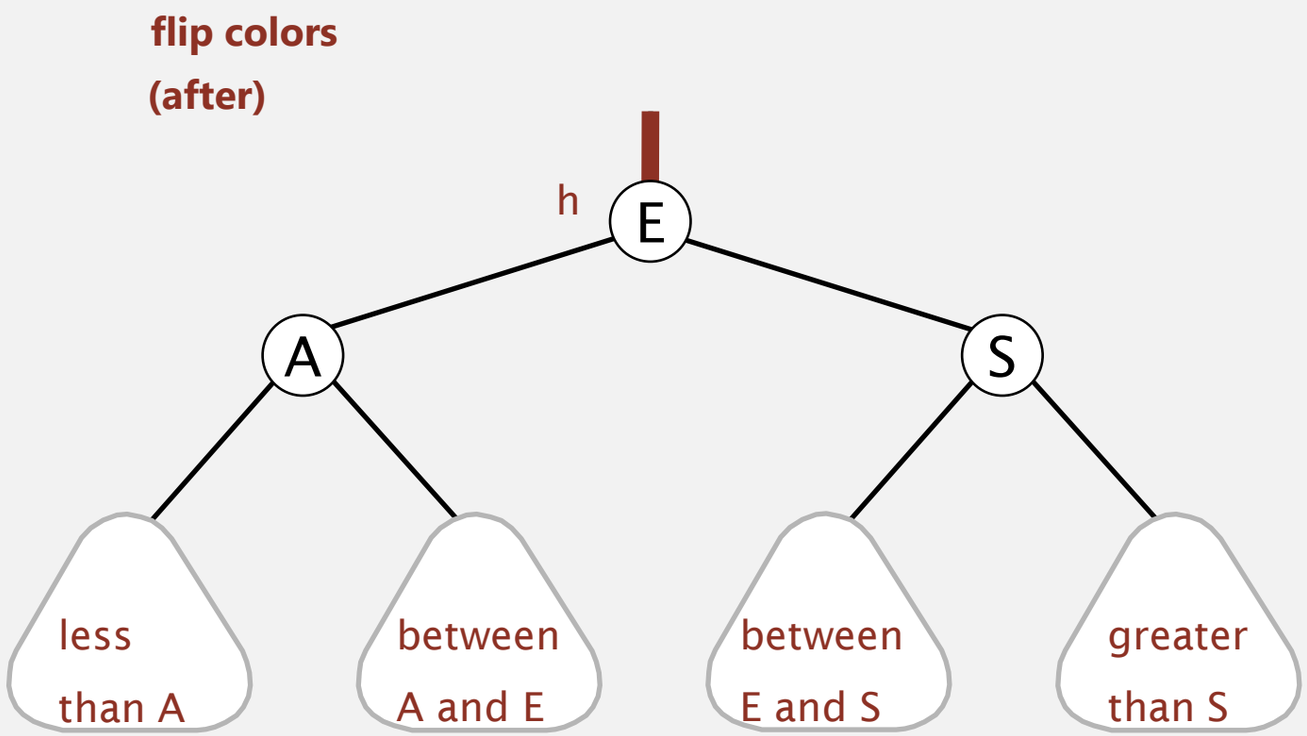



```
private void flipColors(Node h)
{
   assert !isRed(h);
   assert isRed(h.left);
   asset isRed(h.right);
   h.color = RED;
   h.left.color = BLACK;
   h.right.color = BLACK;
}
```

Invariants. Maintains symmetric order and perfect black balance.

Elementary red-black BST operations

Color flip. Recolor to split a (temporary) 4-node.

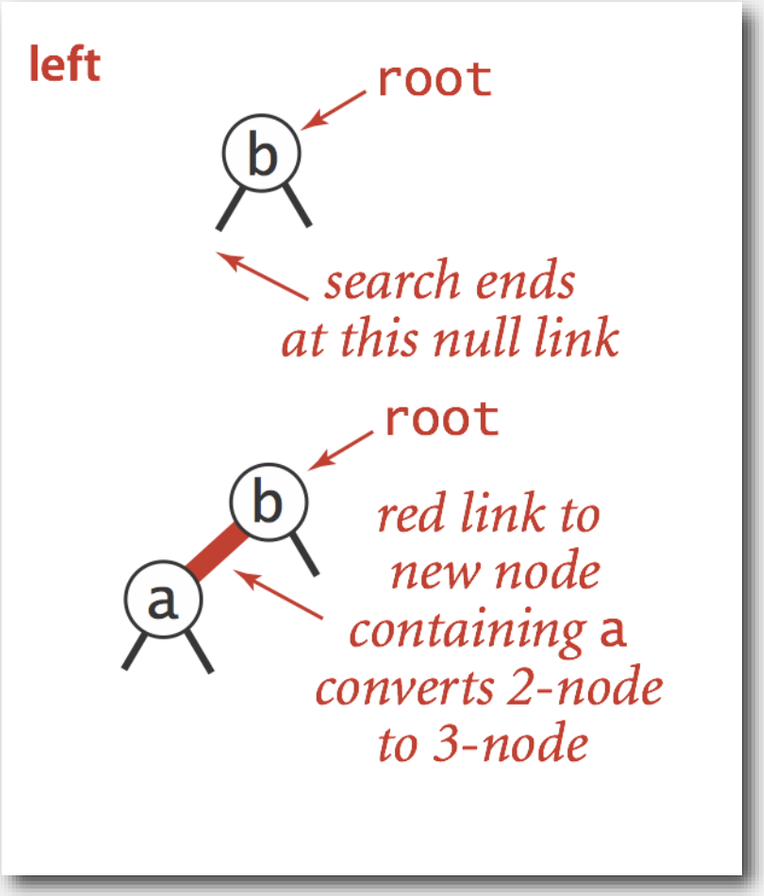


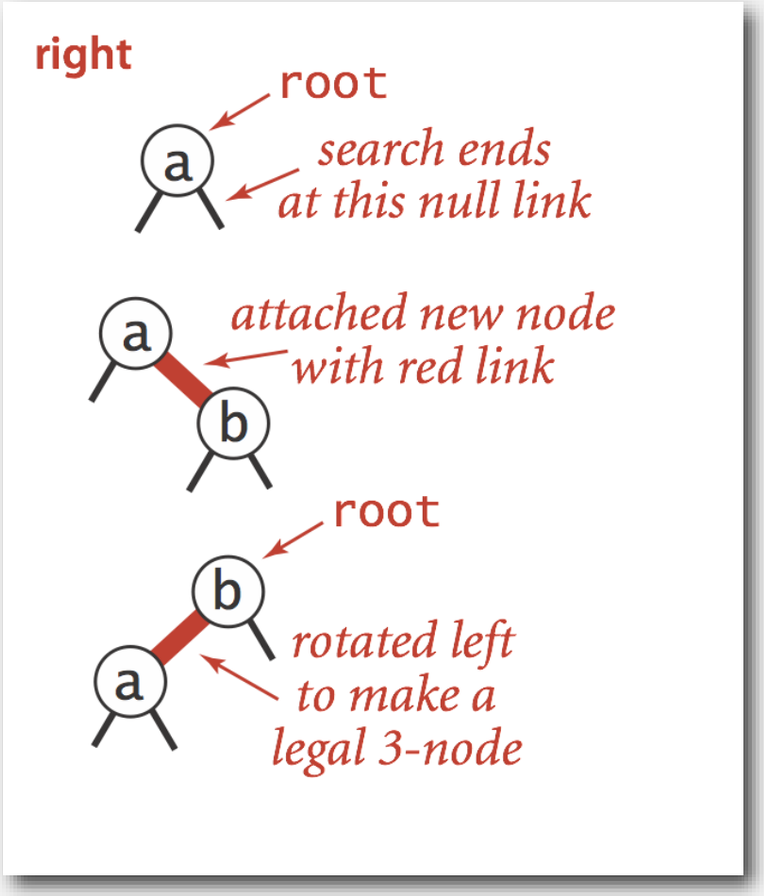

```
private void flipColors(Node h)
{
   assert !isRed(h);
   assert isRed(h.left);
   asset isRed(h.right);
   h.color = RED;
   h.left.color = BLACK;
   h.right.color = BLACK;
}
```

Invariants. Maintains symmetric order and perfect black balance.

Insertion in a LLRB tree

Warmup 1. Insert into a tree with exactly 1 node.

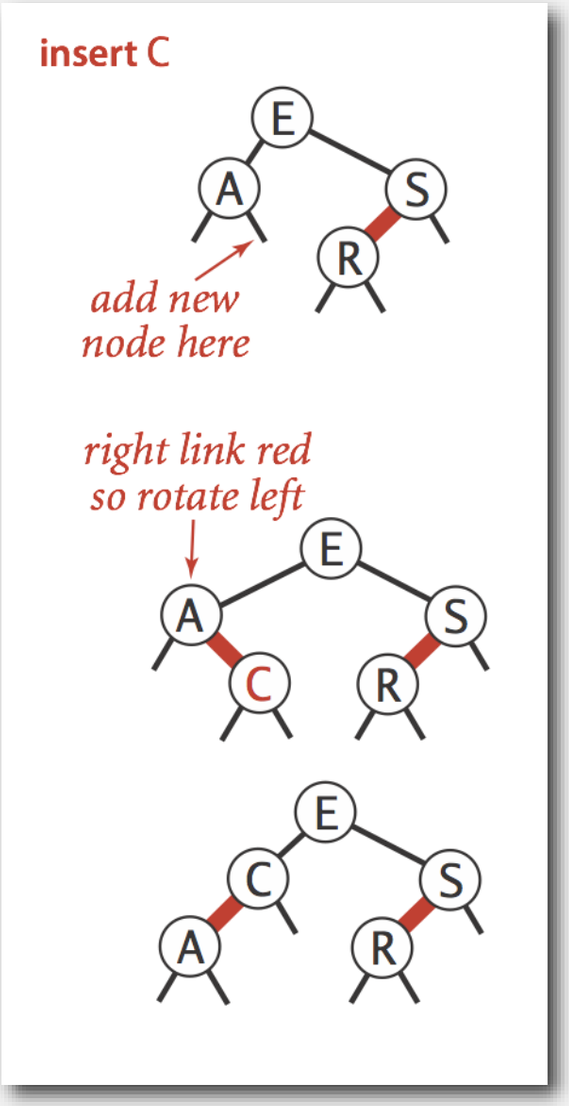




Insertion in a LLRB tree

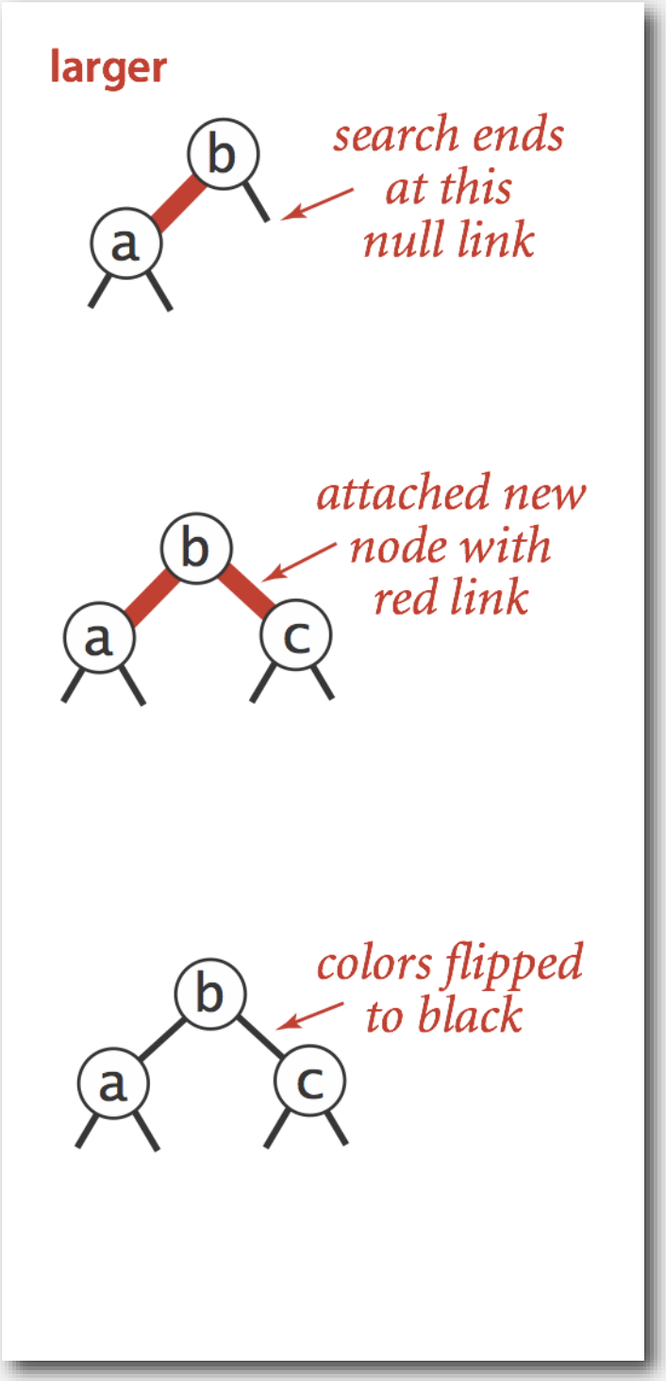
Case 1. Insert into a 2-node at the bottom.

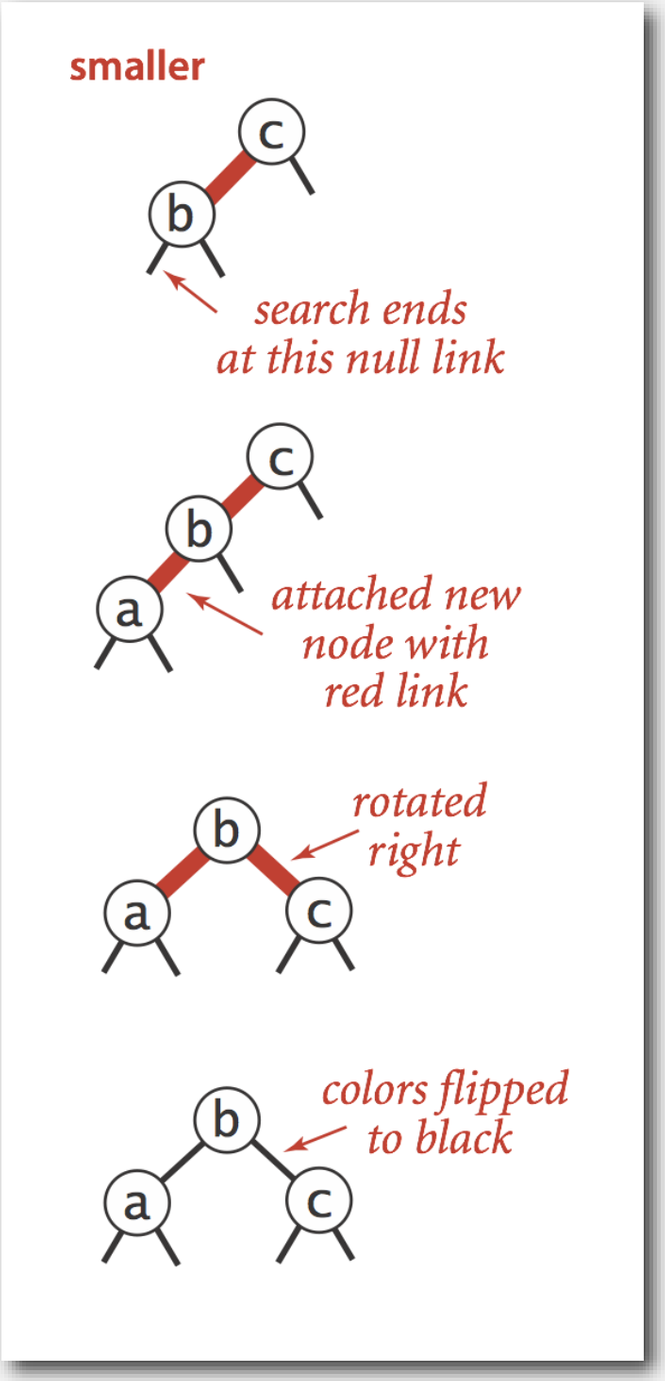
- Do standard BST insert; color new link red.
- If new red link is a right link, rotate left.

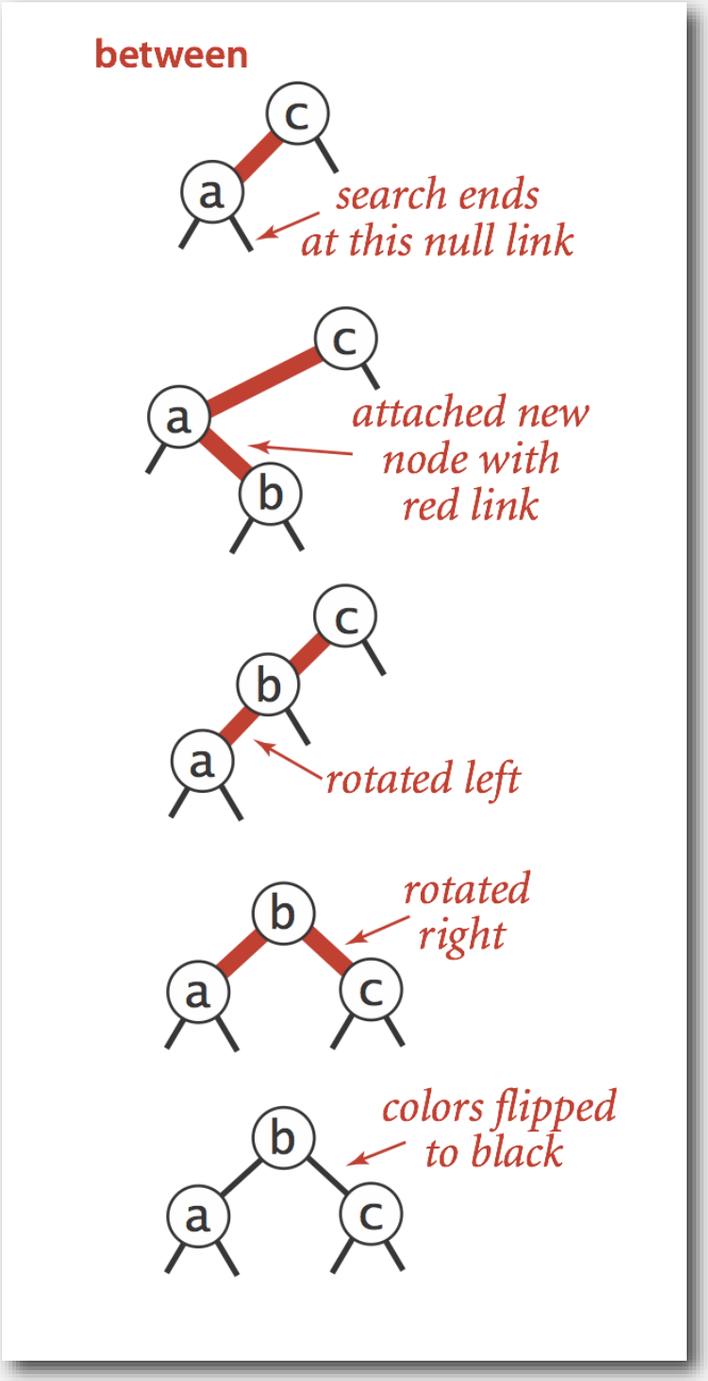


Insertion in a LLRB tree

Warmup 2. Insert into a tree with exactly 2 nodes.



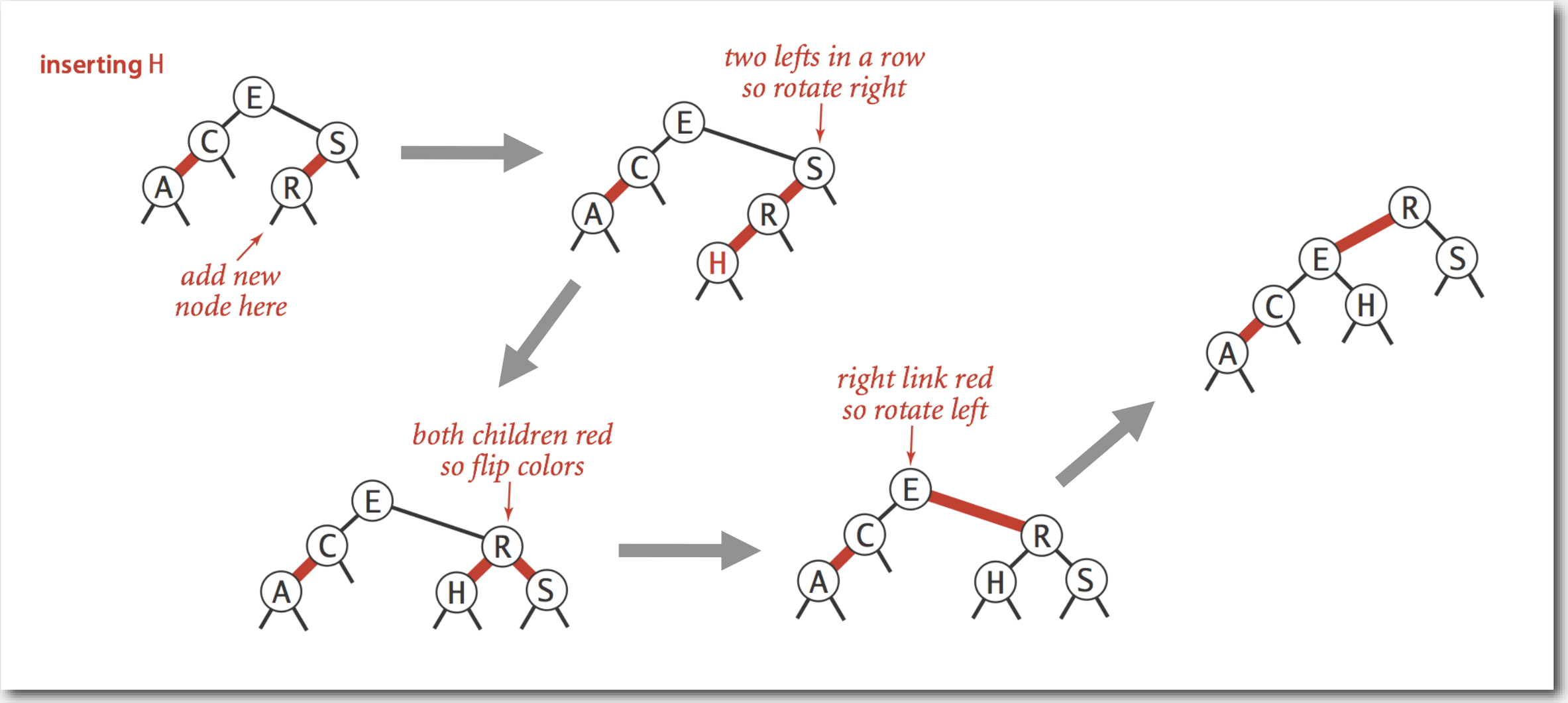




Insertion in a LLRB tree

Case 2. Insert into a 3-node at the bottom.

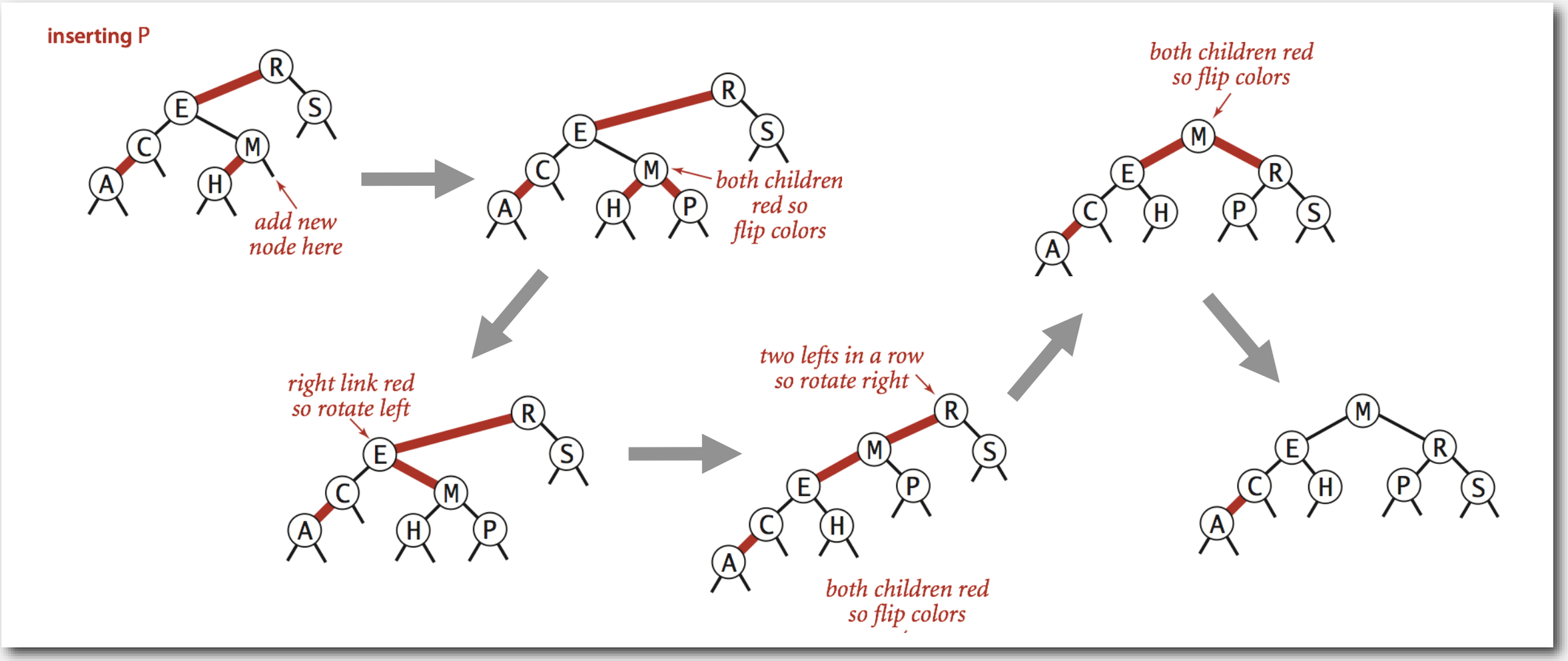
- Do standard BST insert; color new link red.
- Rotate to balance the 4-node (if needed).
- Flip colors to pass red link up one level.
- Rotate to make lean left (if needed).



Insertion in a LLRB tree: passing red links up the tree

Case 2. Insert into a 3-node at the bottom.

- Do standard BST insert; color new link red.
- Rotate to balance the 4-node (if needed).
- Flip colors to pass red link up one level.
- Rotate to make lean left (if needed).
- Repeat case 1 or case 2 up the tree (if needed).

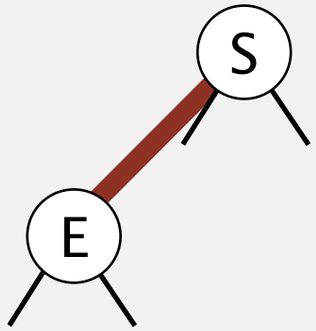


Red-black BST insertion

insert S

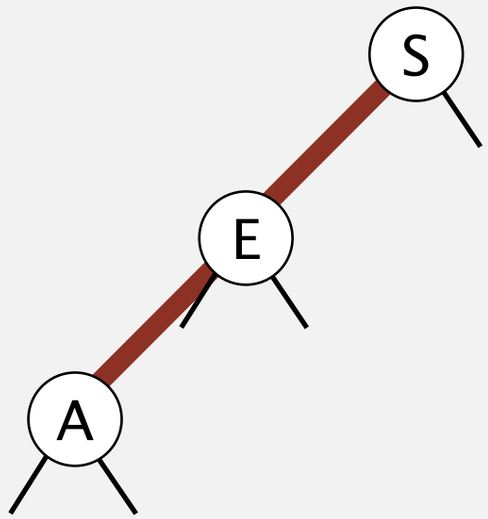
Red-black BST insertion

insert E



Red-black BST insertion

insert A

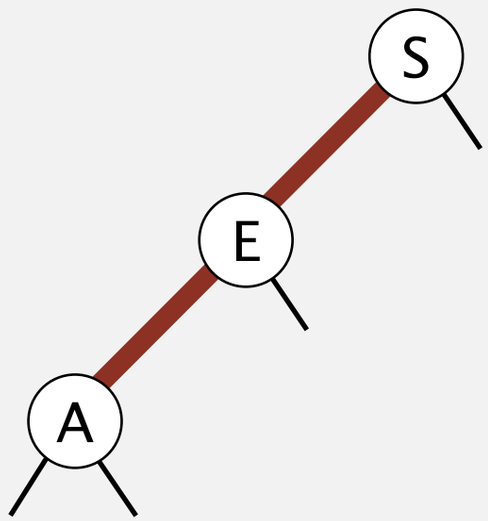


Red-black BST insertion

insert A

two left reds in a row

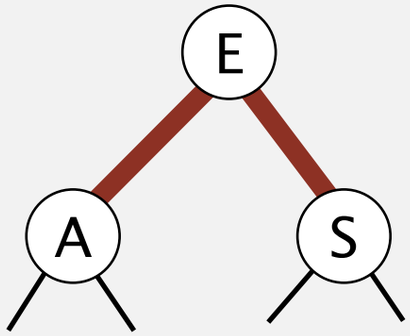
(rotate S right)



Red-black BST insertion

both children red

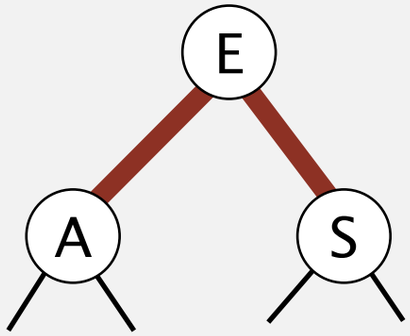
(flip colors)



Red-black BST insertion

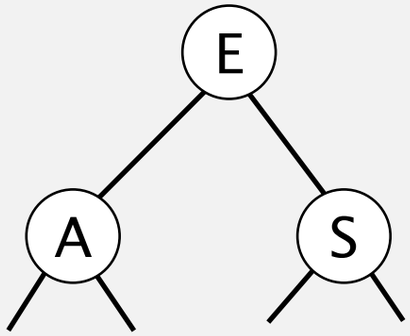
both children red

(flip colors)



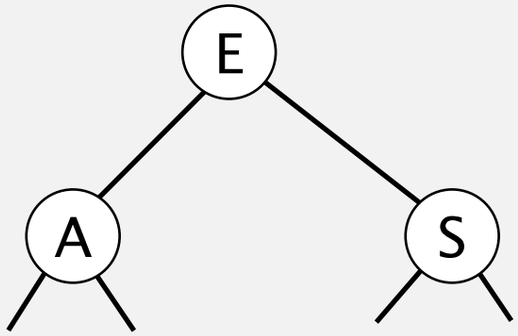
Red-black BST insertion

red-black BST



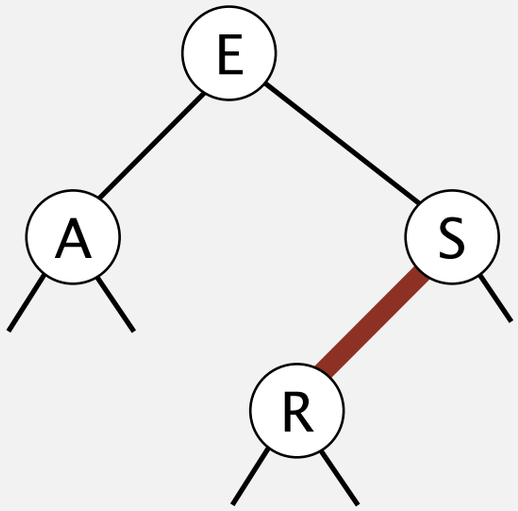
Red-black BST insertion

red-black BST



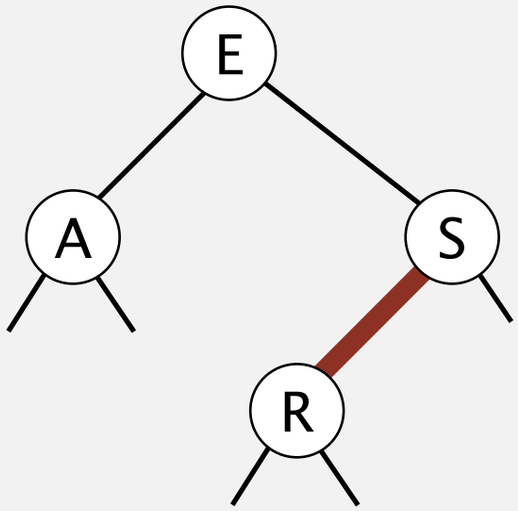
Red-black BST insertion

insert R



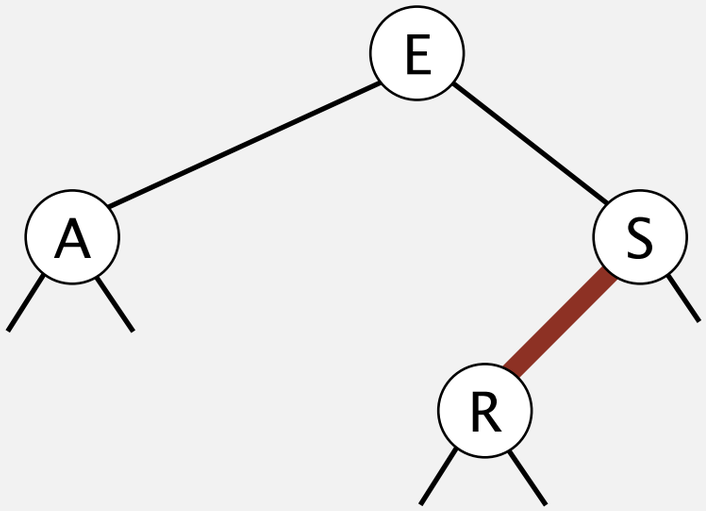
Red-black BST insertion

red-black BST



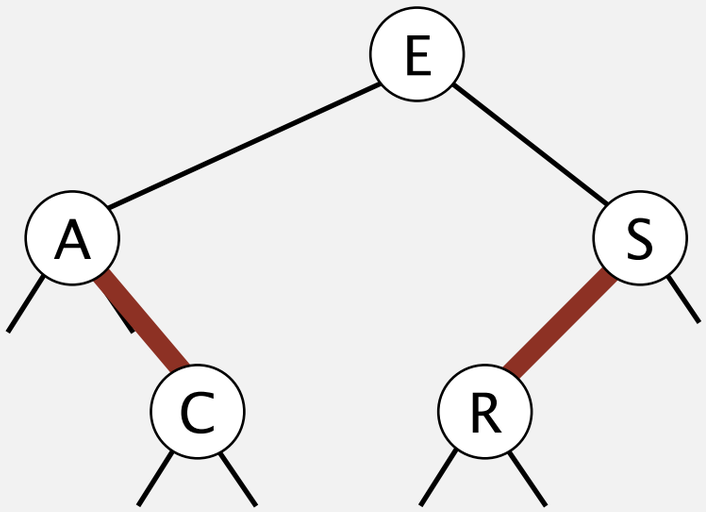
Red-black BST insertion

red-black BST

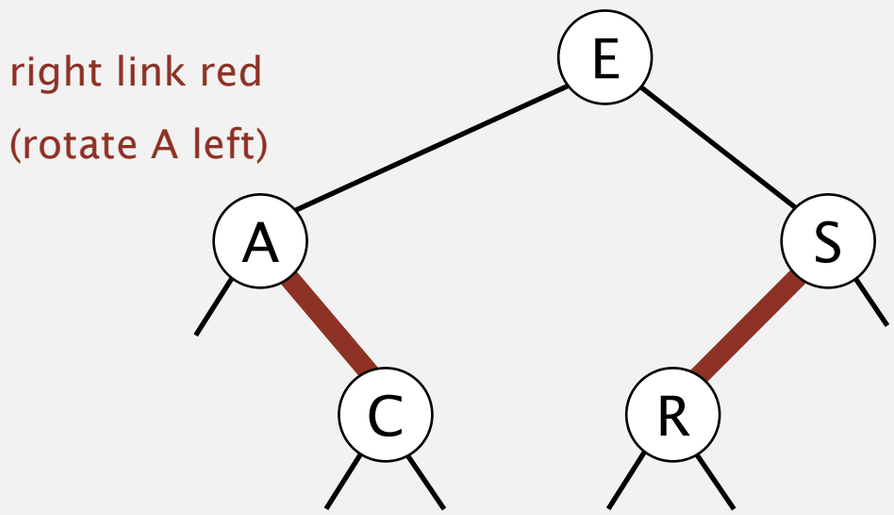


Red-black BST insertion

insert C

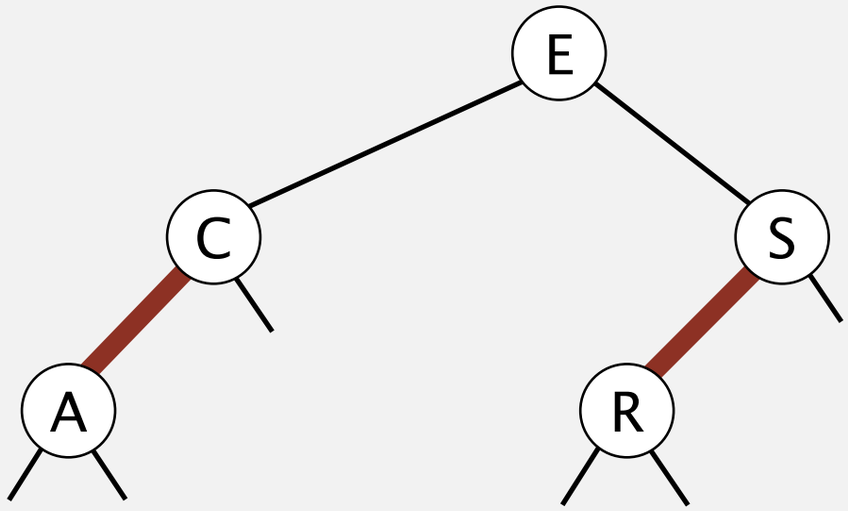


Red-black BST insertion



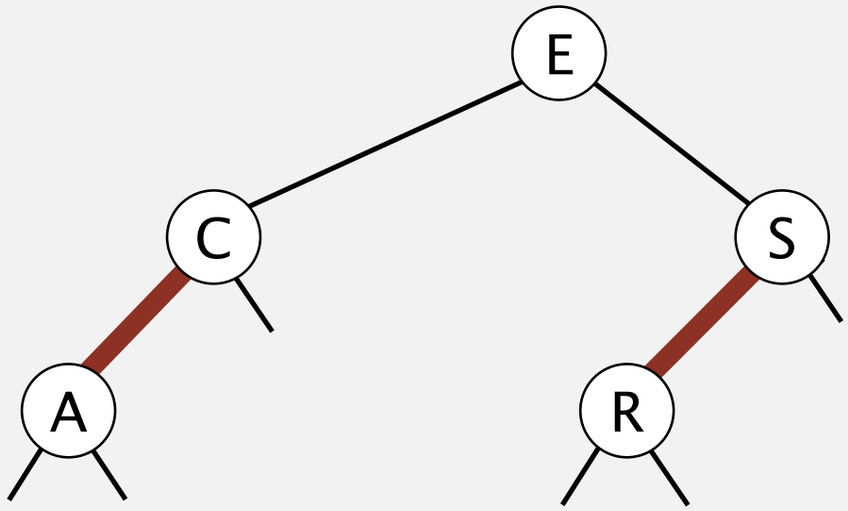
Red-black BST insertion

red-black BST



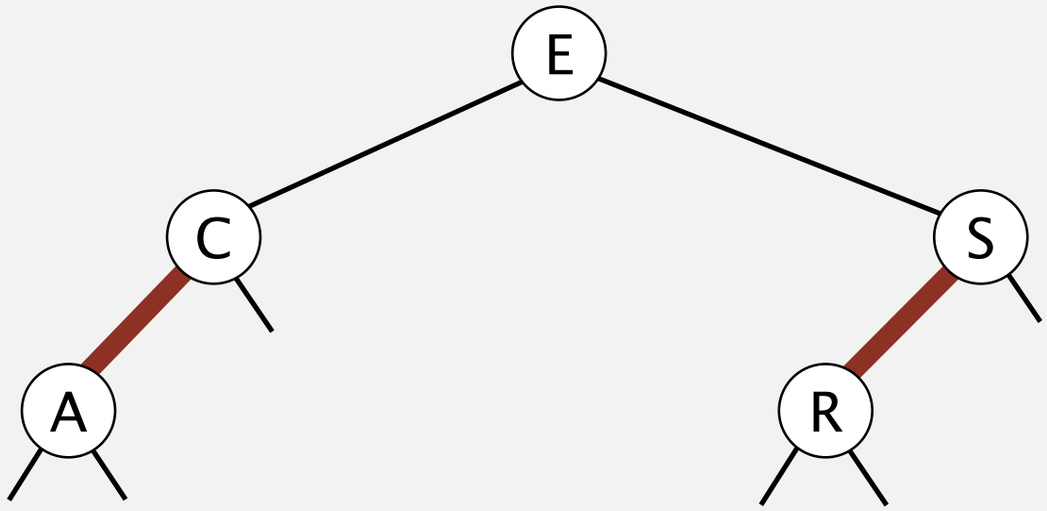
Red-black BST insertion

red-black BST



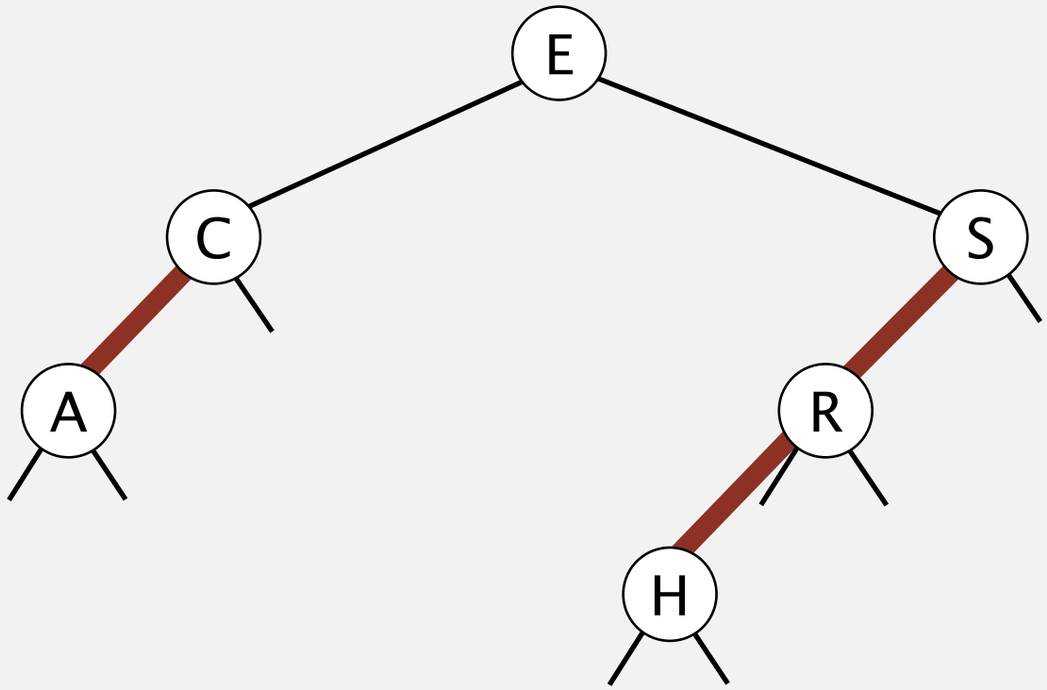
Red-black BST insertion

red-black BST

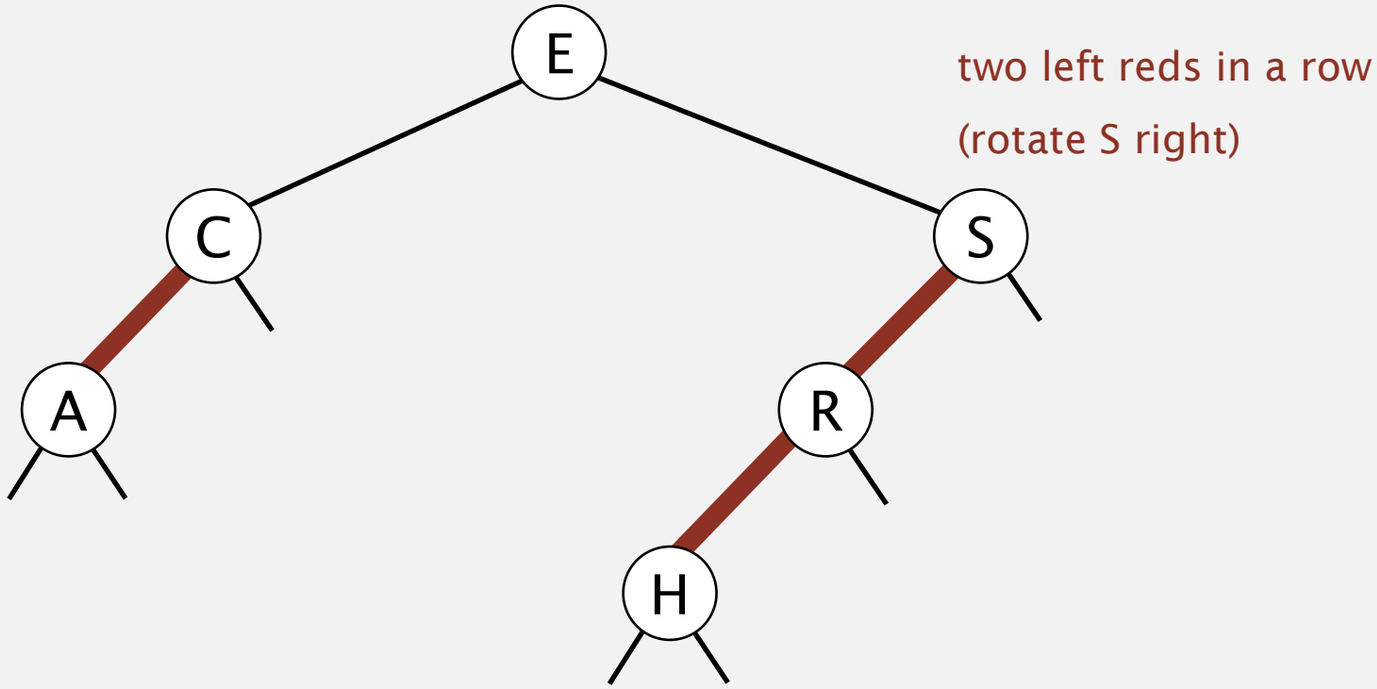


Red-black BST insertion

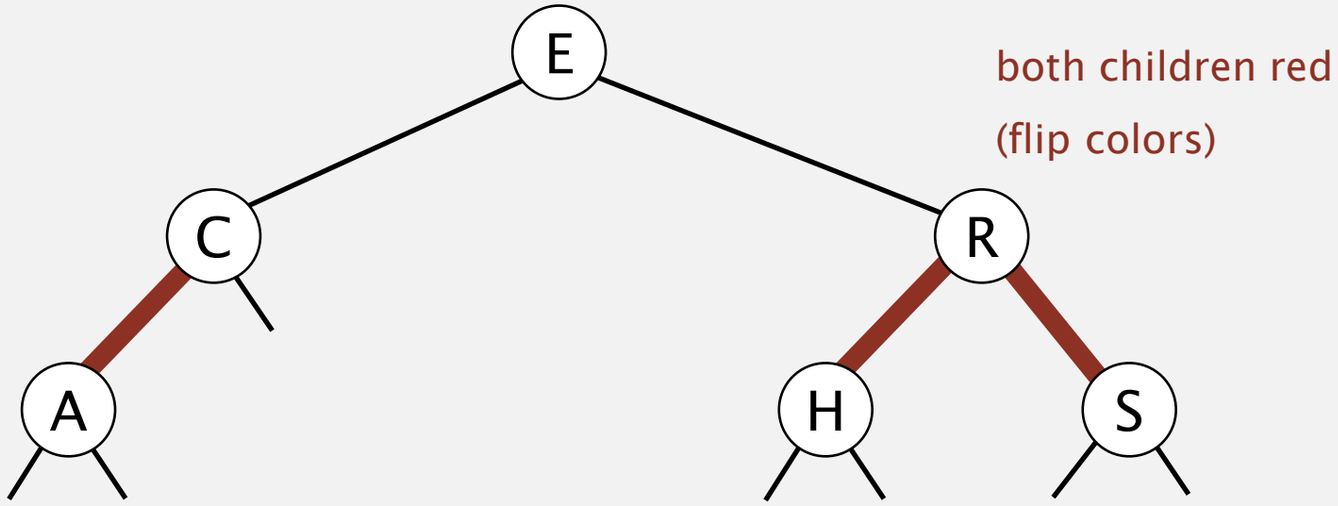
insert H



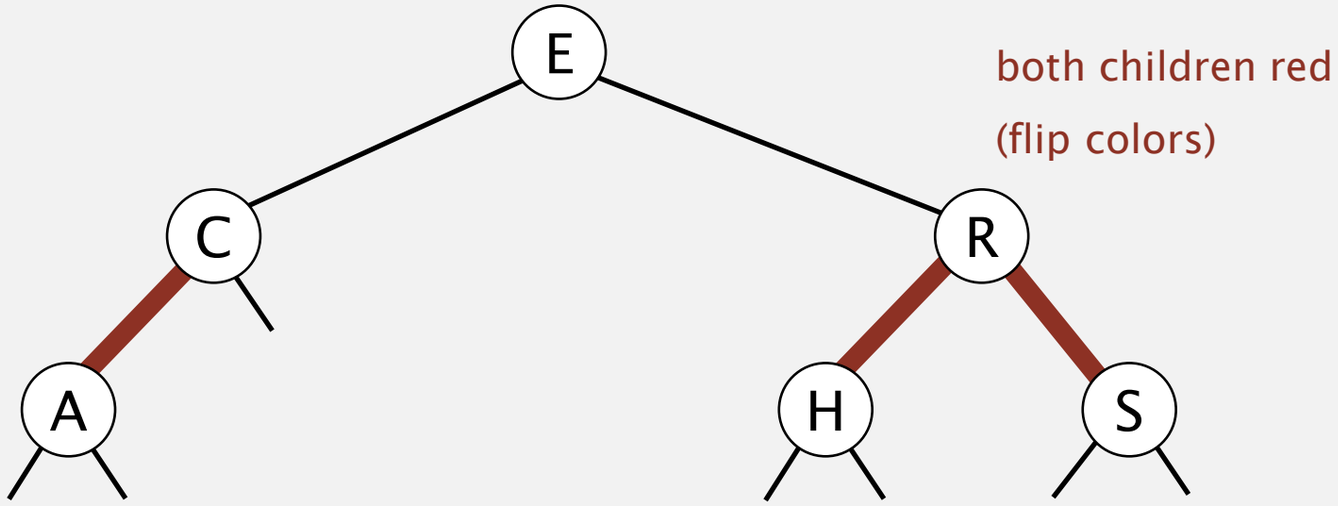
Red-black BST insertion



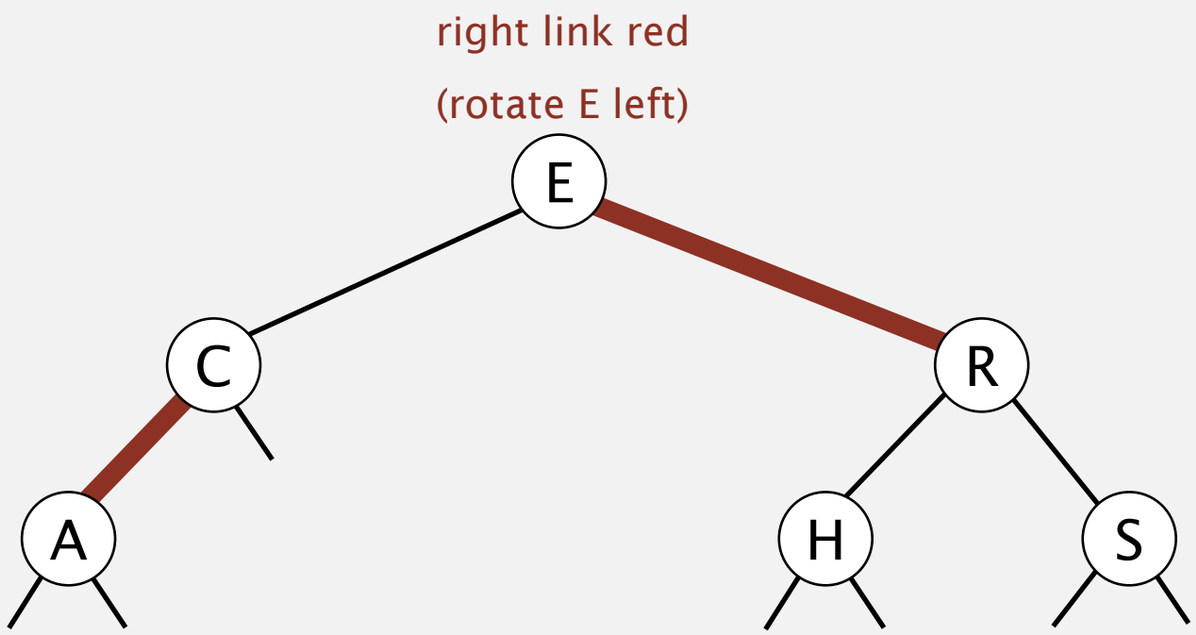
Red-black BST insertion



Red-black BST insertion

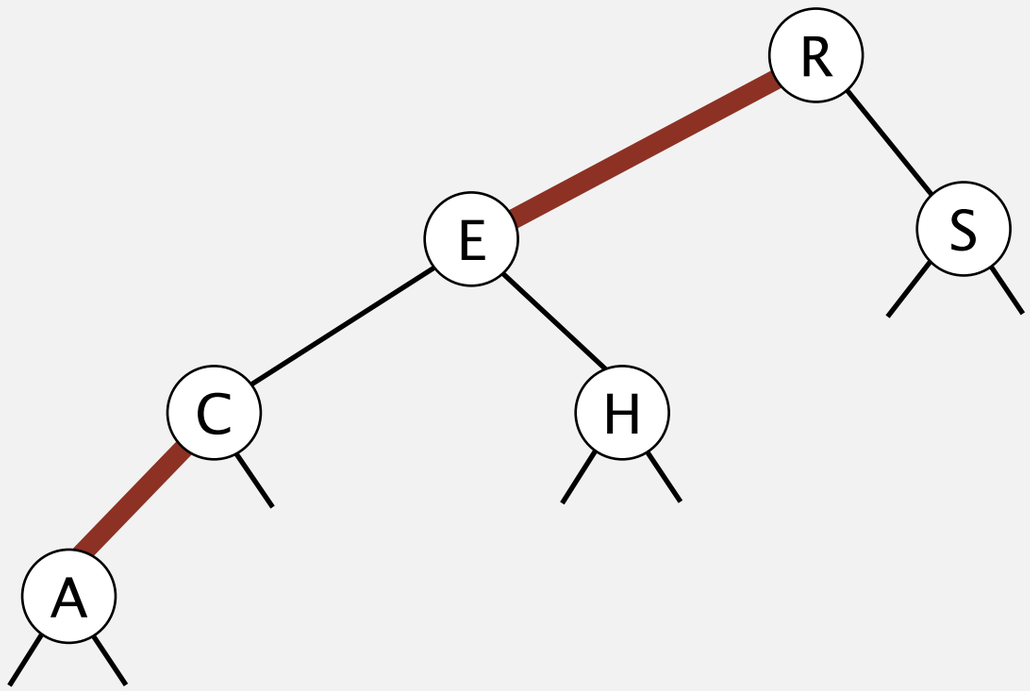


Red-black BST insertion



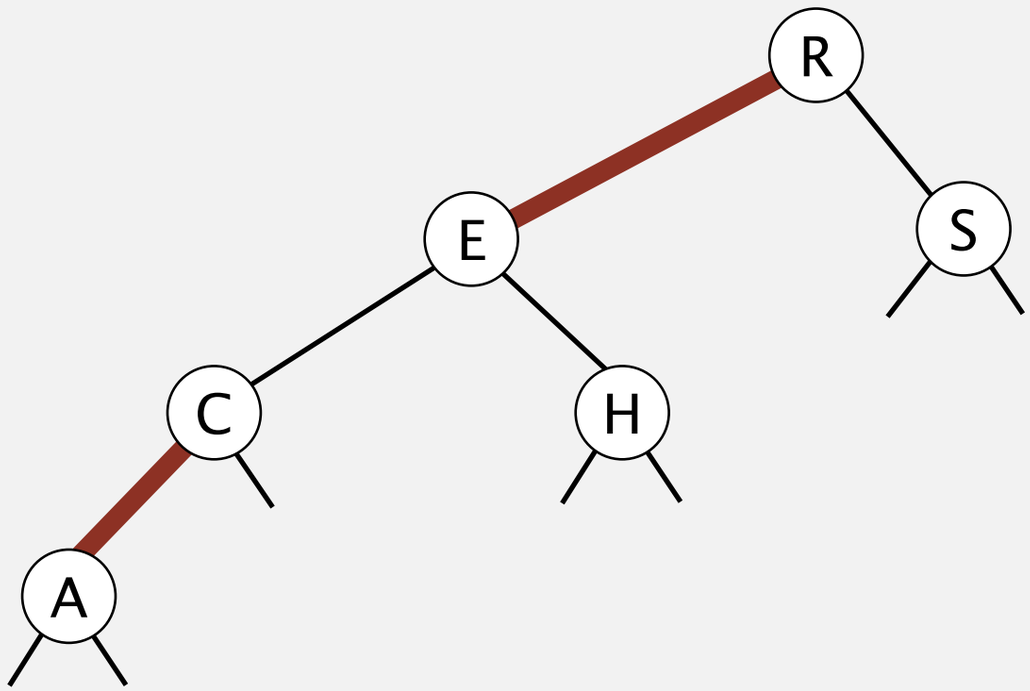
Red-black BST insertion

red-black BST



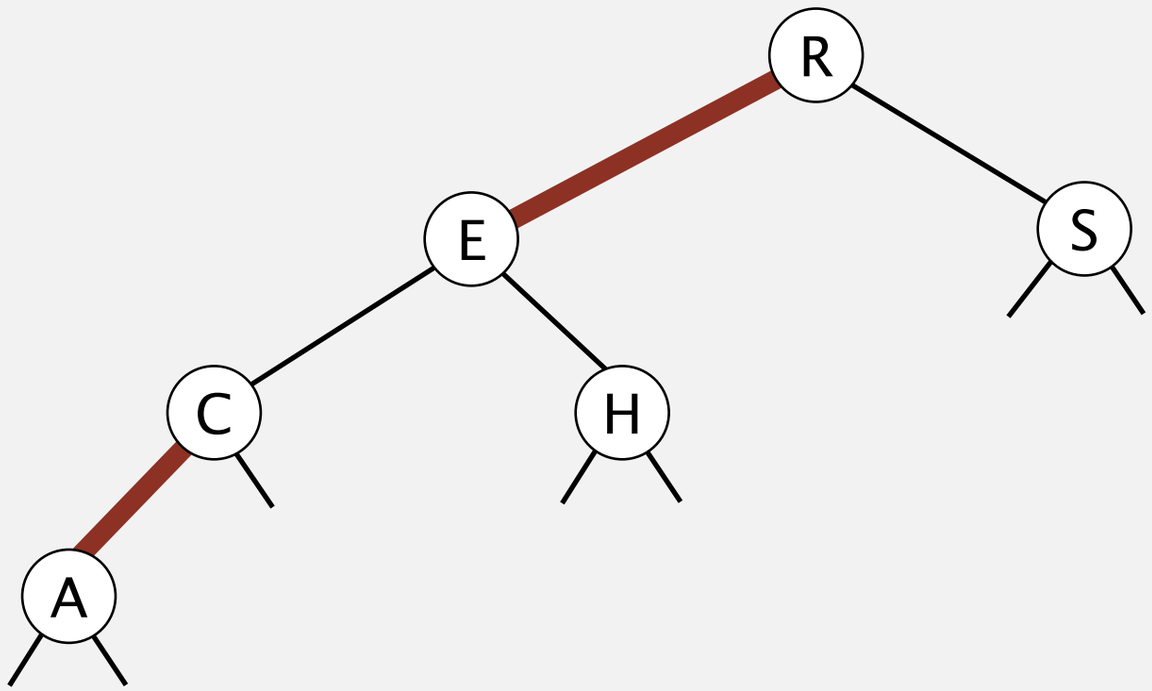
Red-black BST insertion

red-black BST



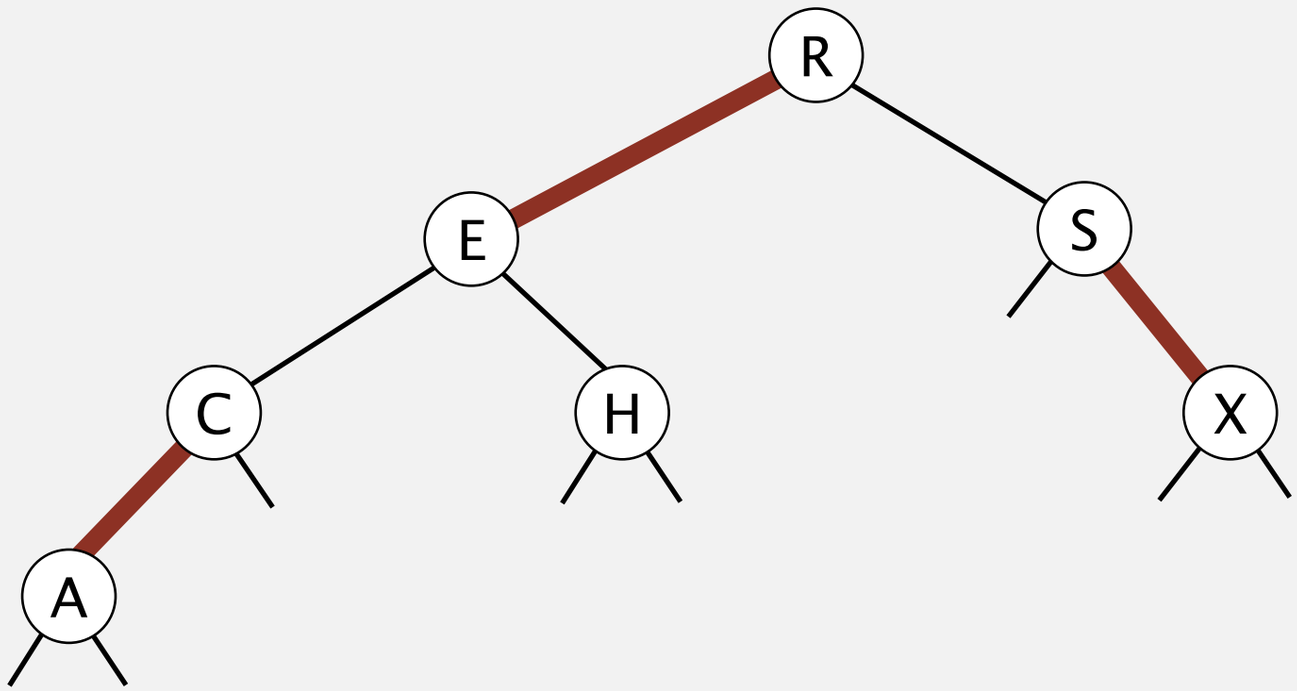
Red-black BST insertion

red-black BST



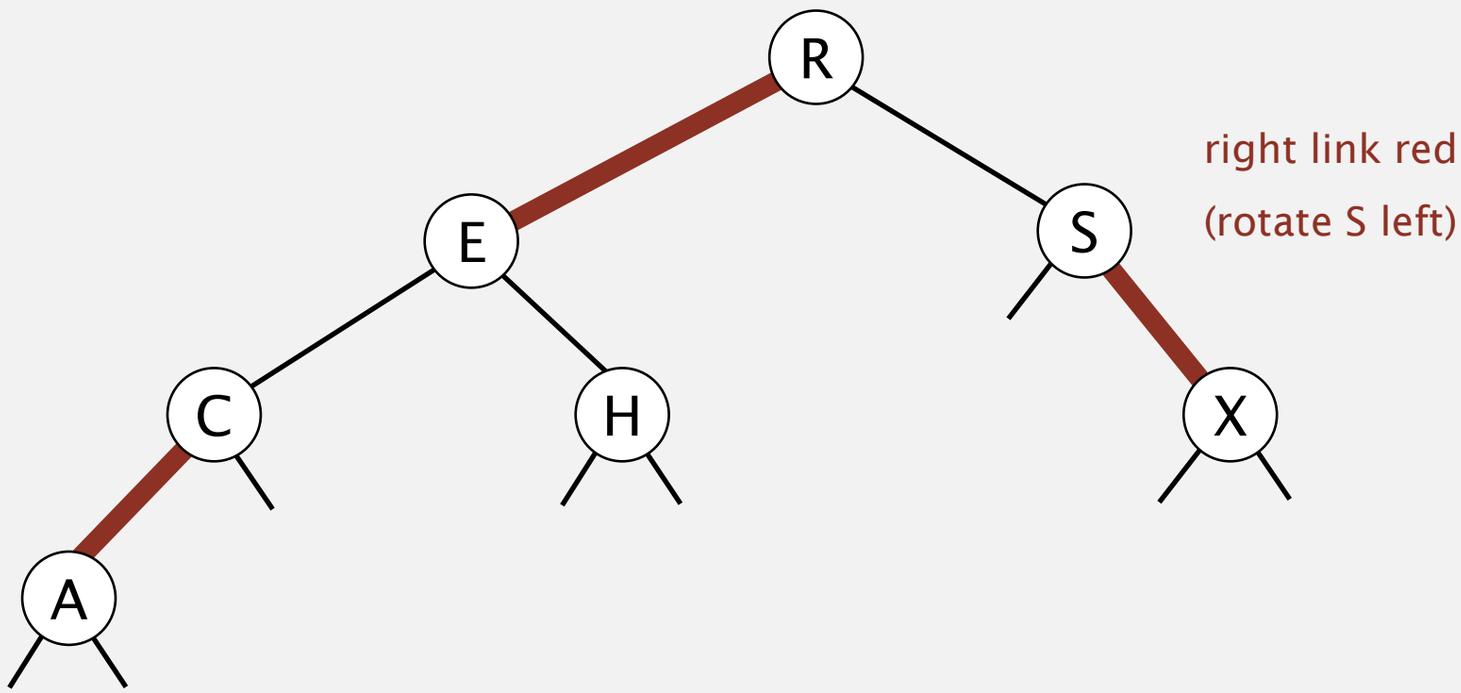
Red-black BST insertion

insert X



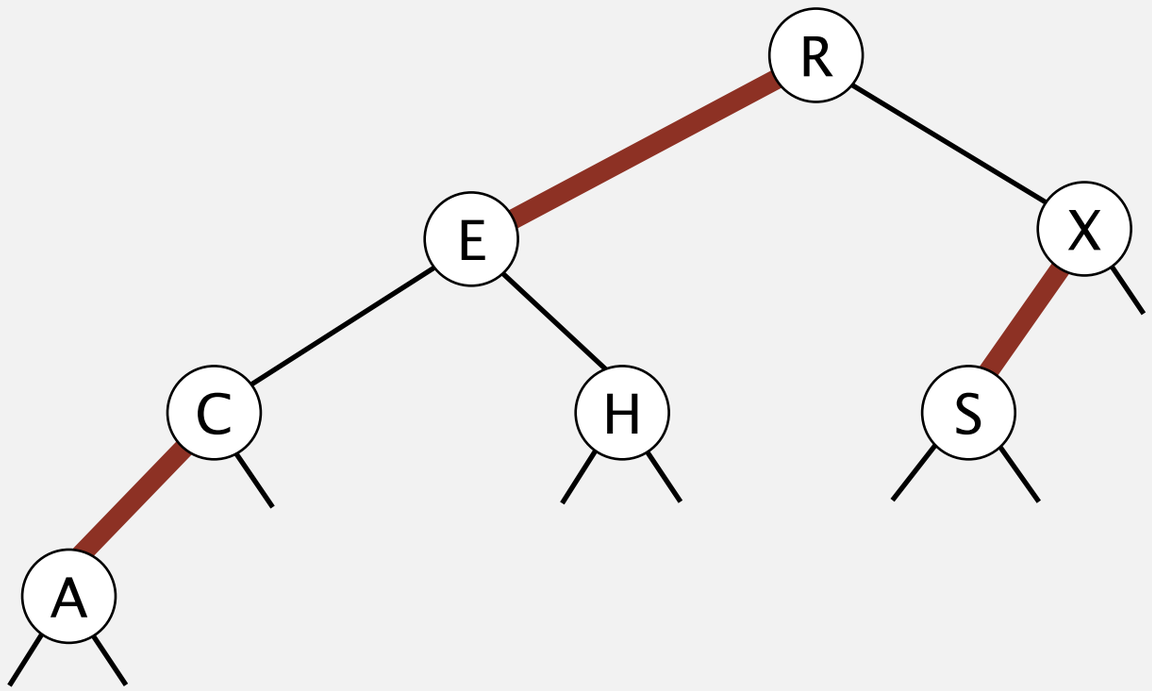
Red-black BST insertion

insert X



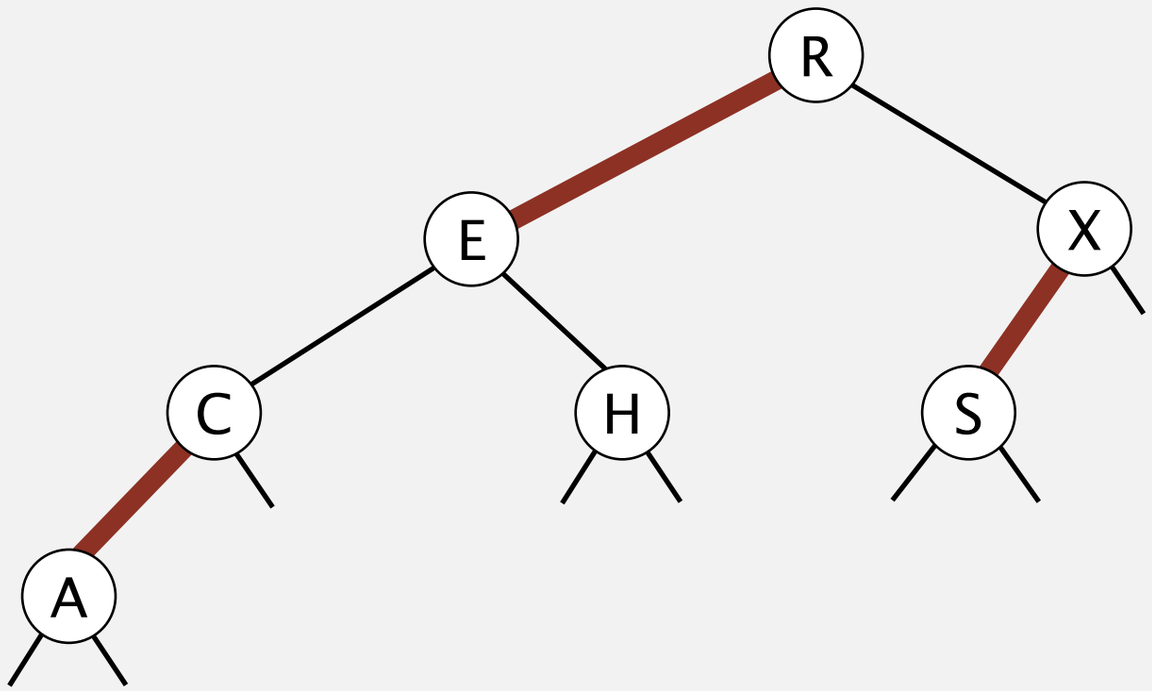
Red-black BST insertion

red-black BST



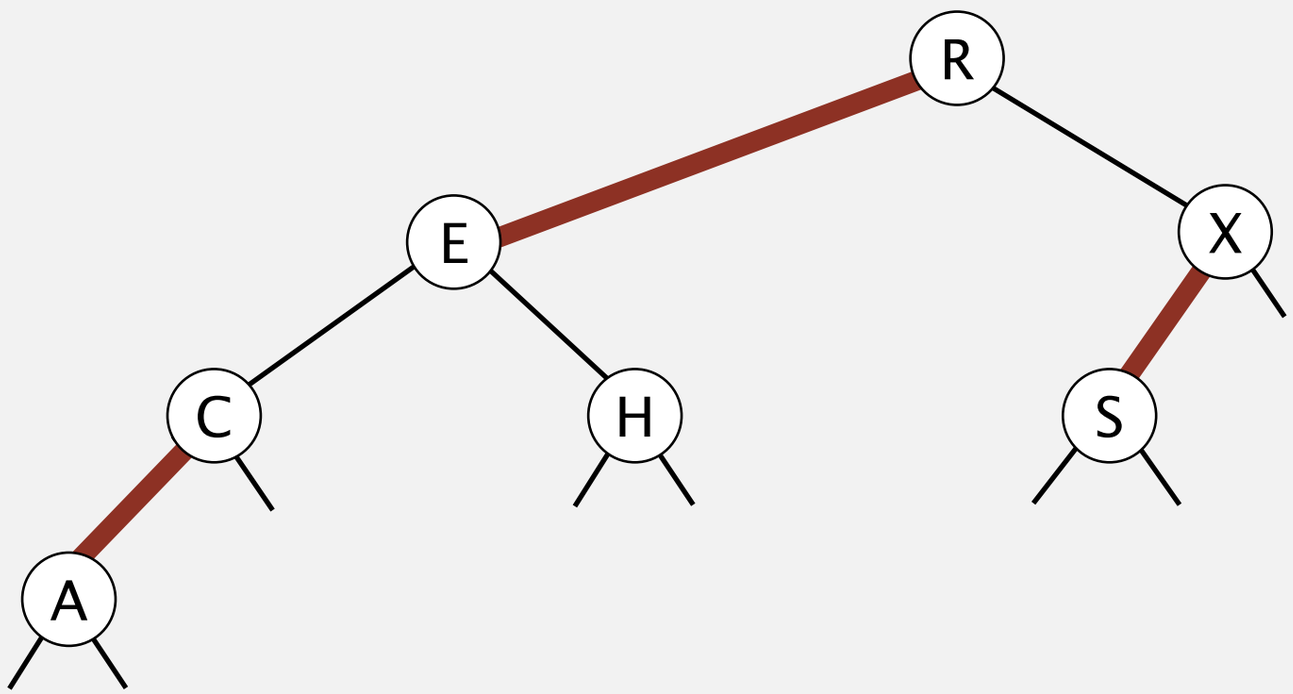
Red-black BST insertion

red-black BST



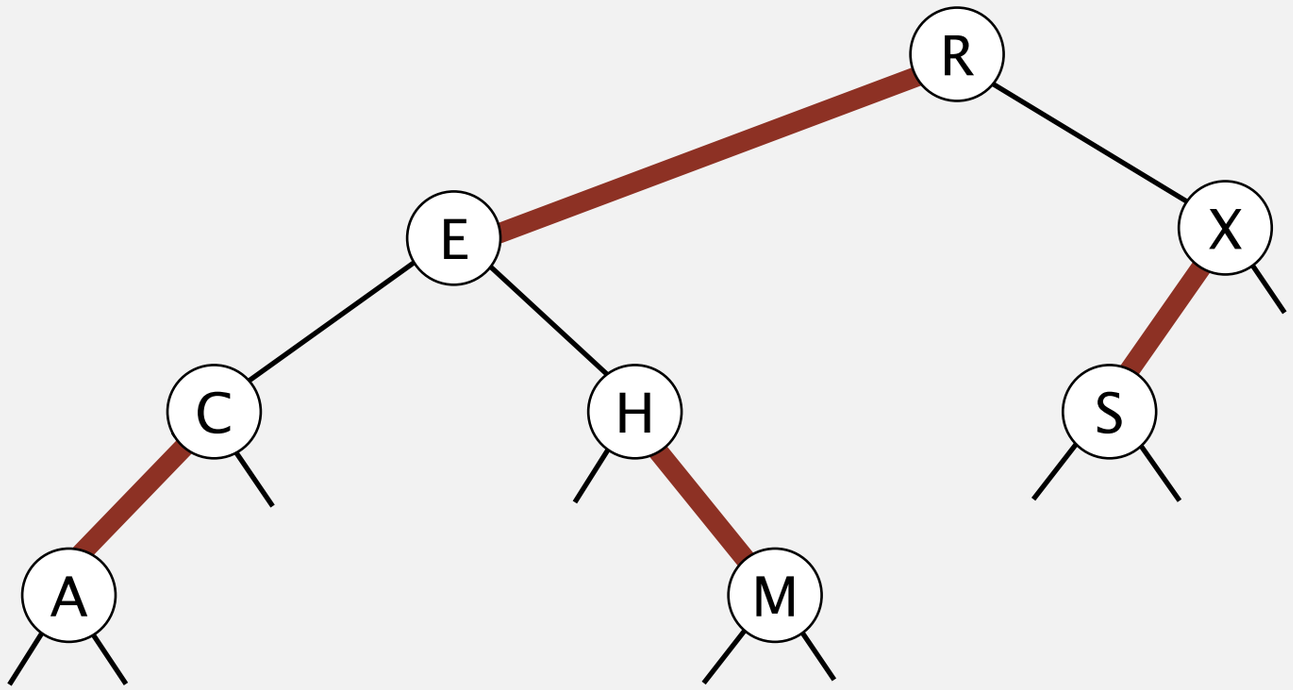
Red-black BST insertion

red-black BST



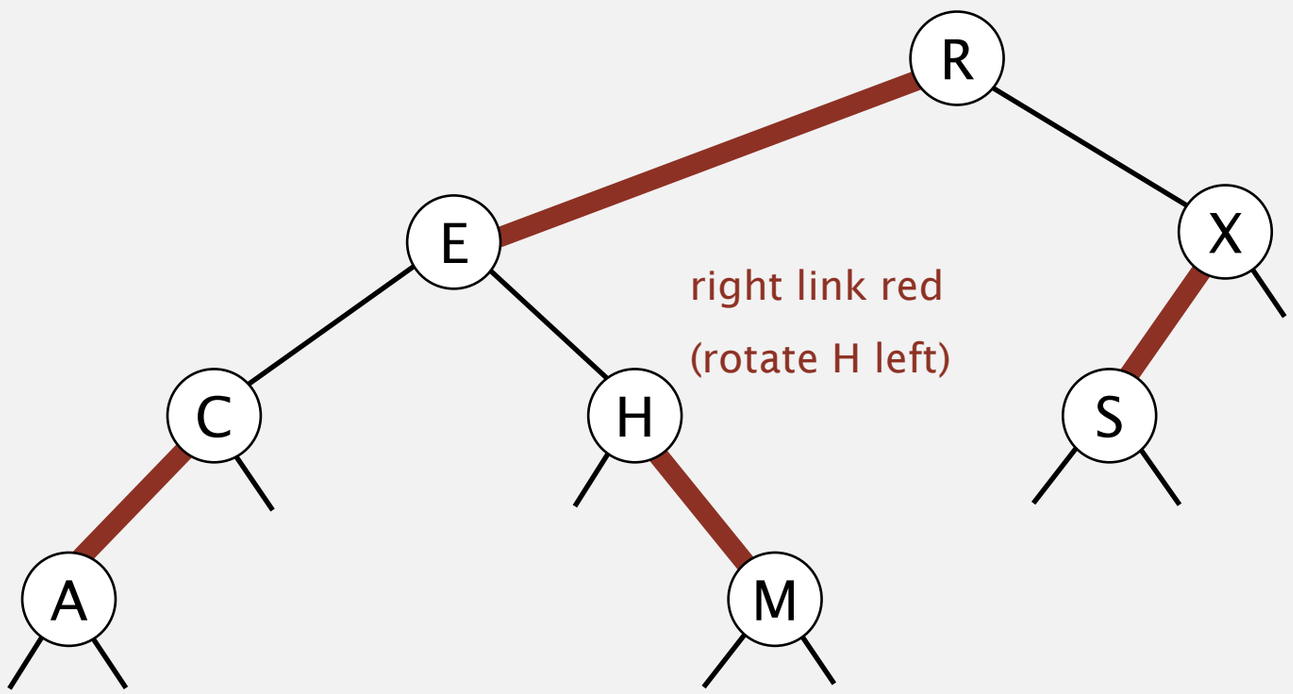
Red-black BST insertion

insert M



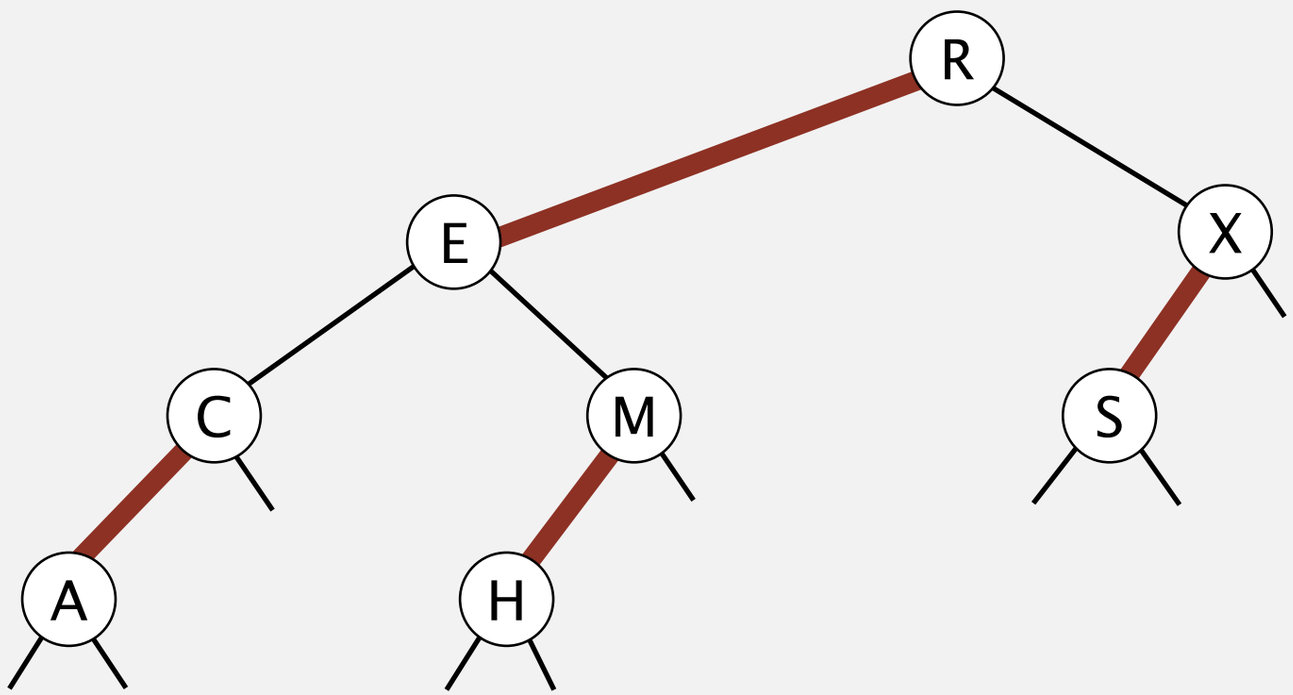
Red-black BST insertion

insert M



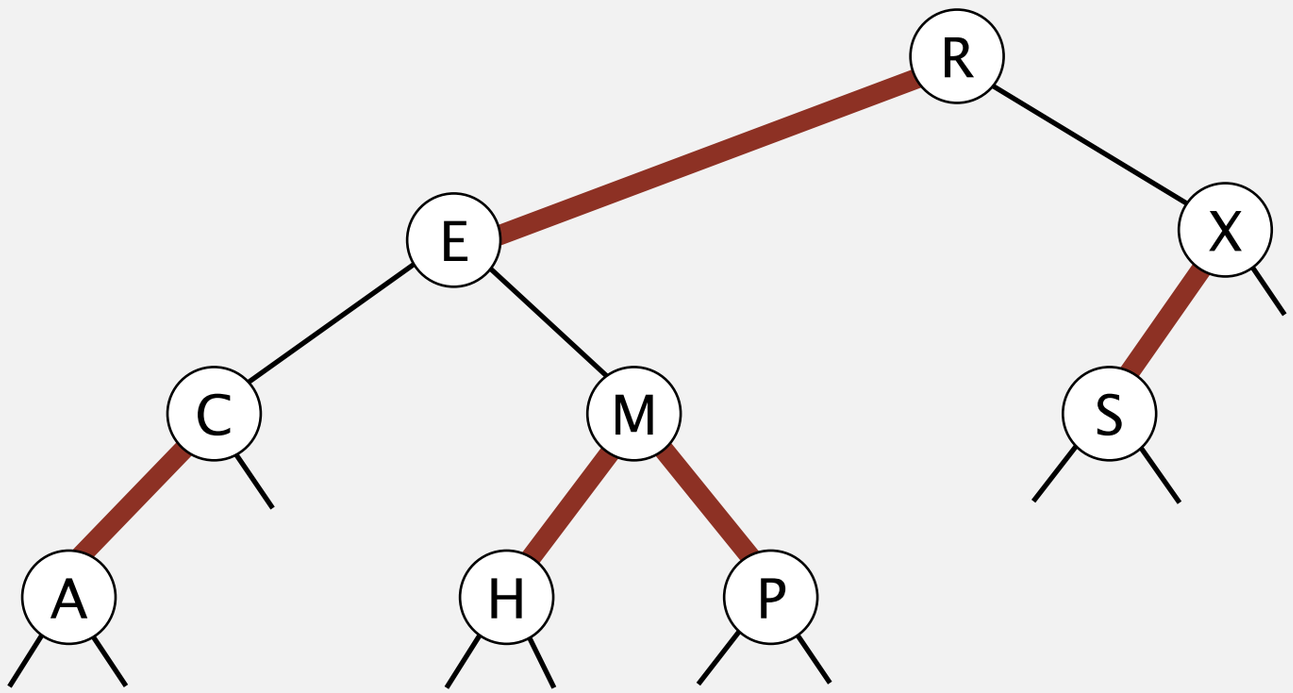
Red-black BST insertion

red-black BST



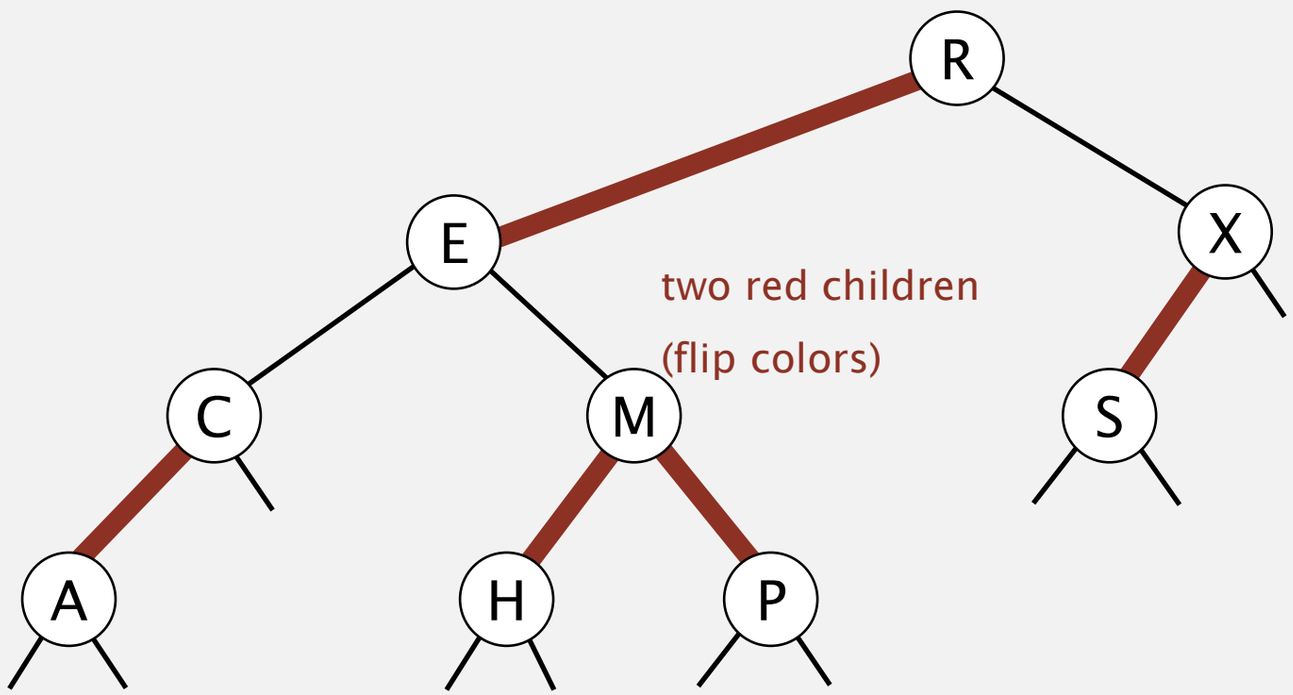
Red-black BST insertion

insert P



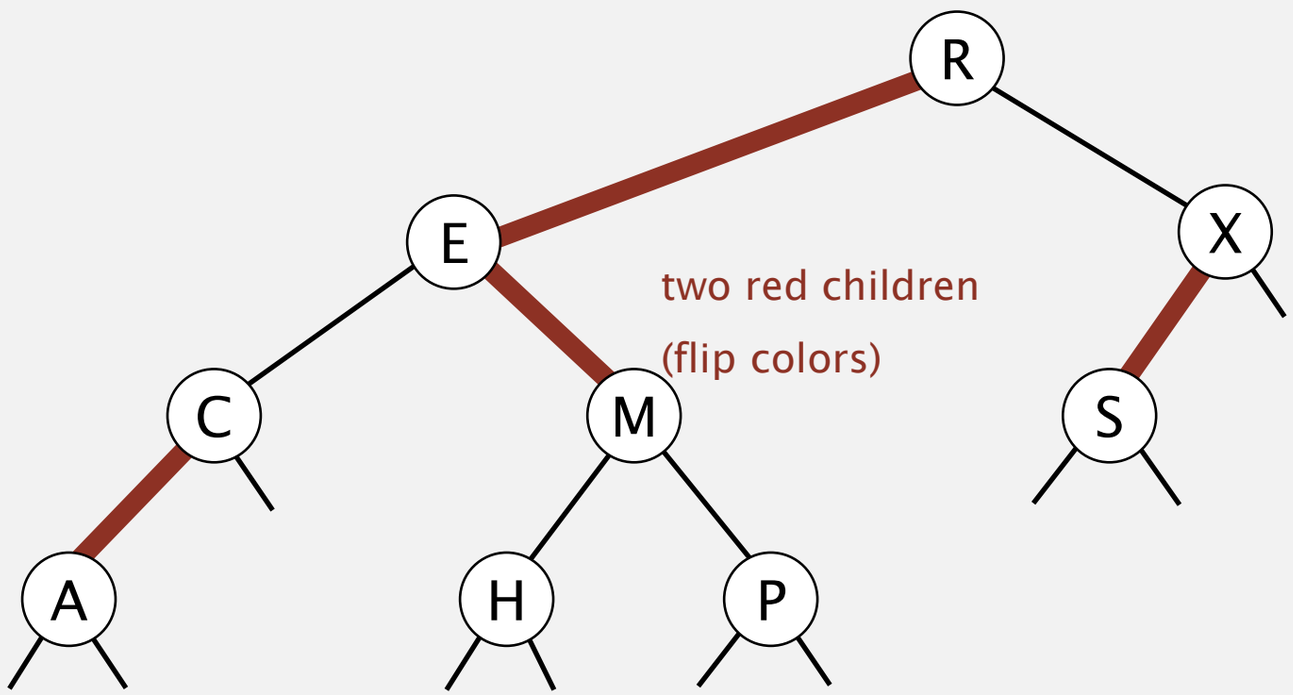
Red-black BST insertion

insert P

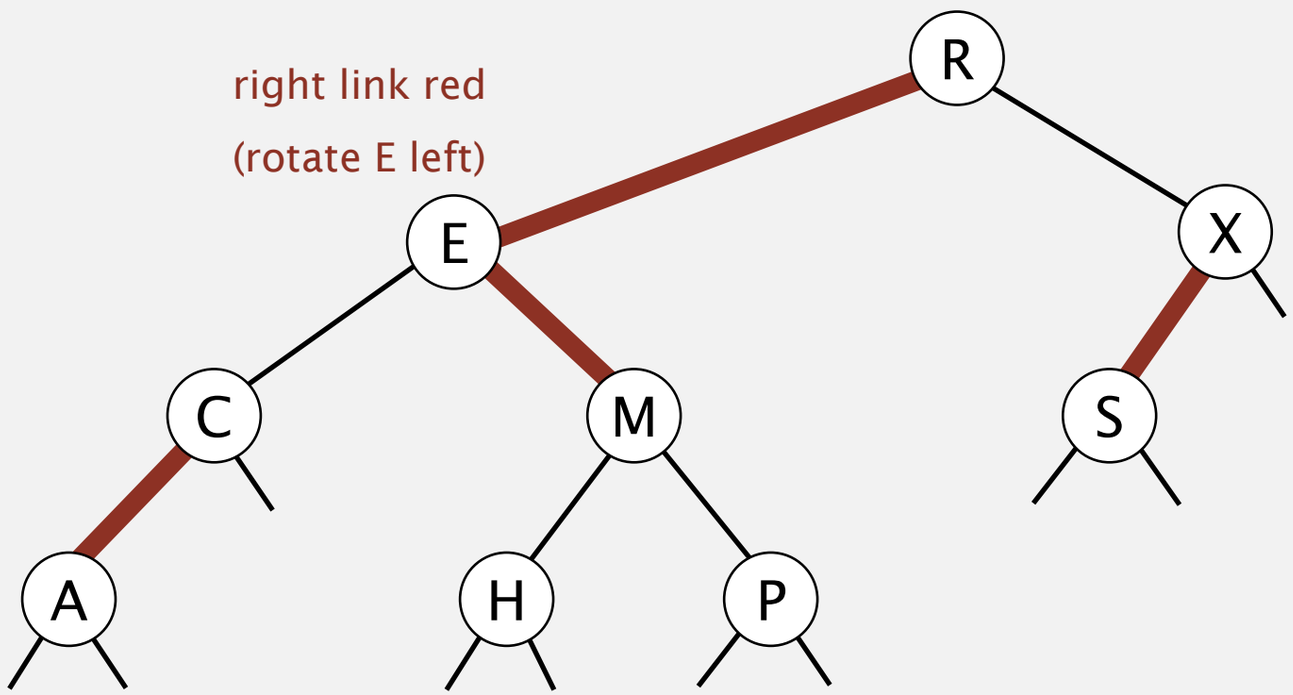


Red-black BST insertion

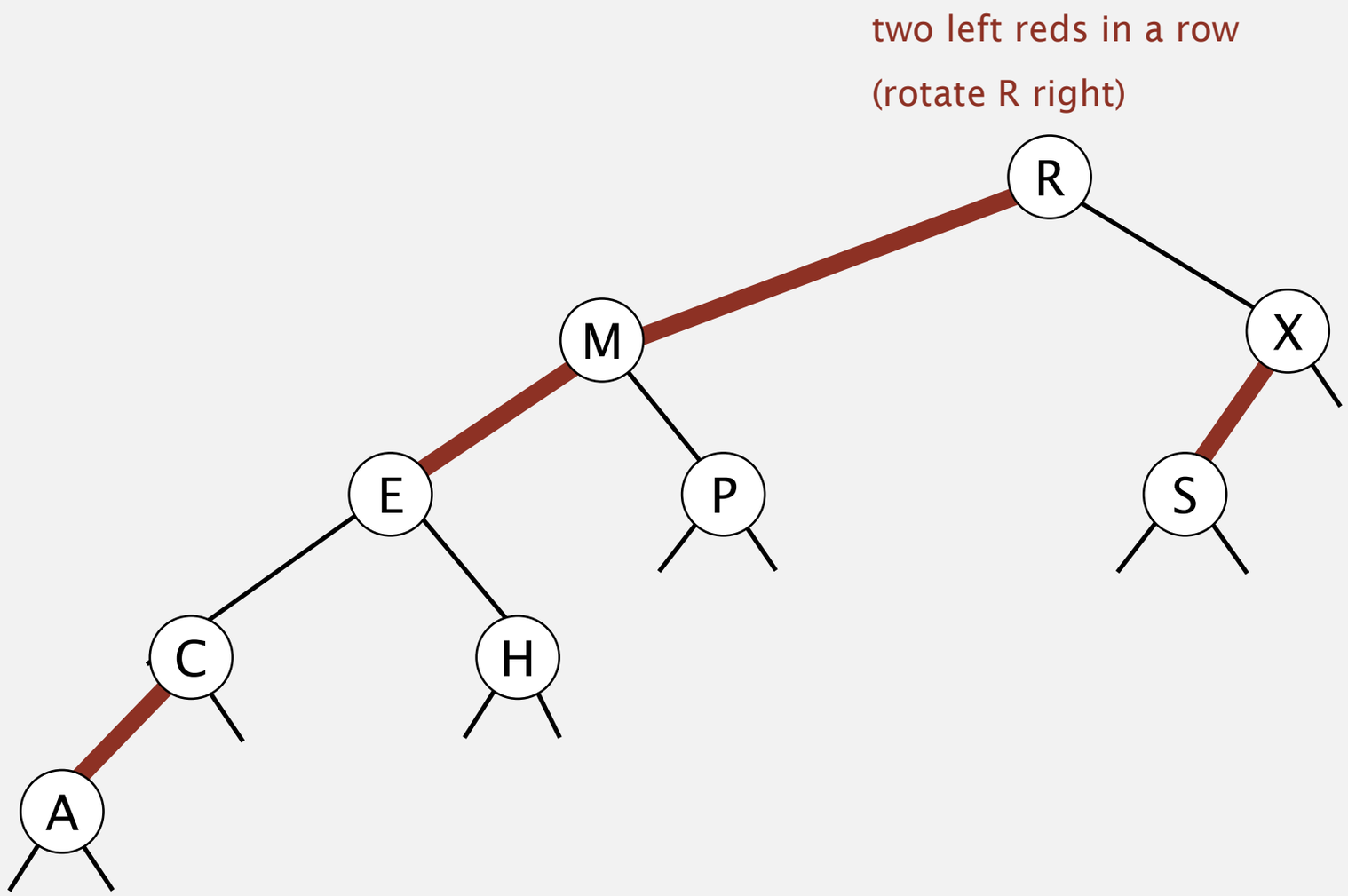
insert P



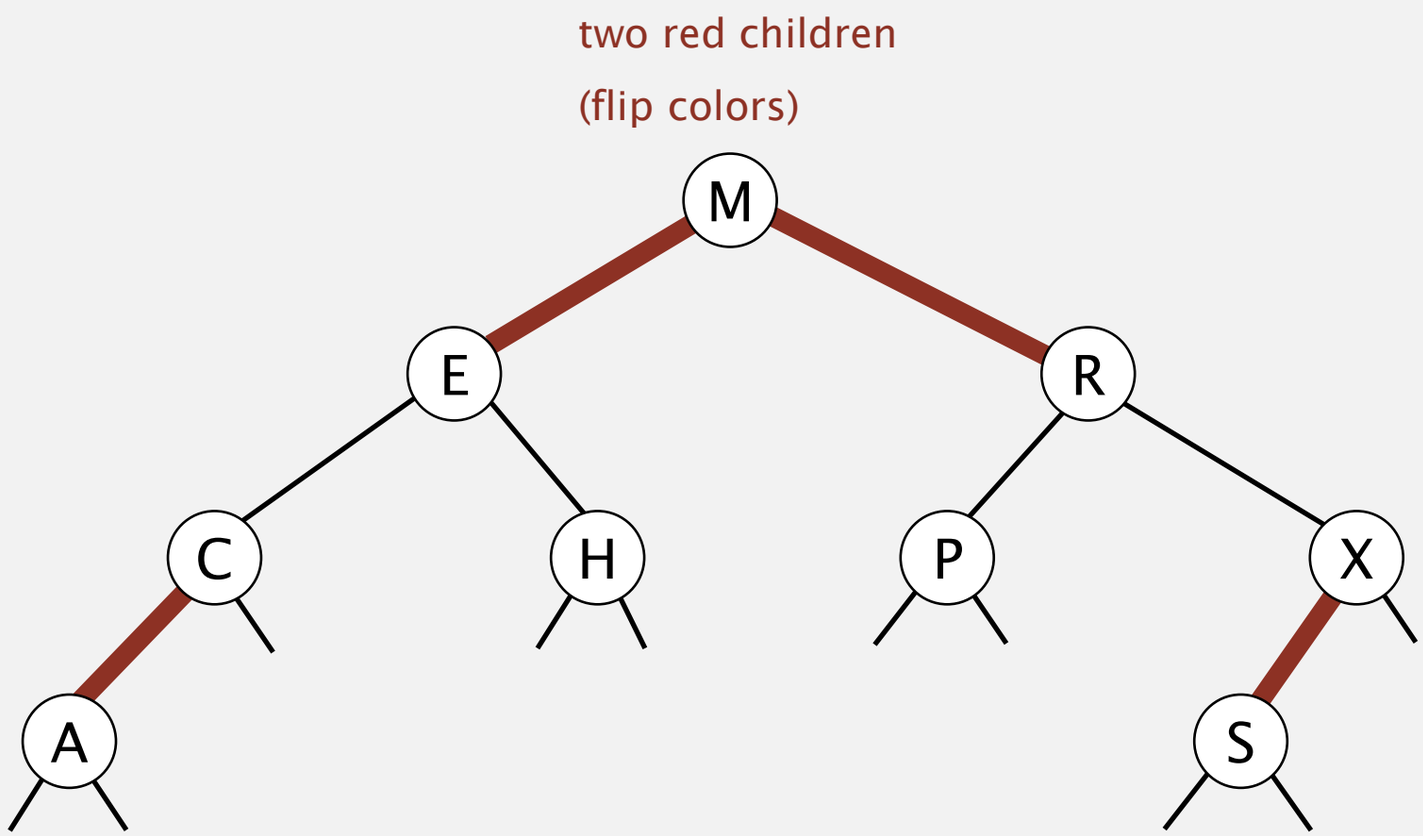
Red-black BST insertion



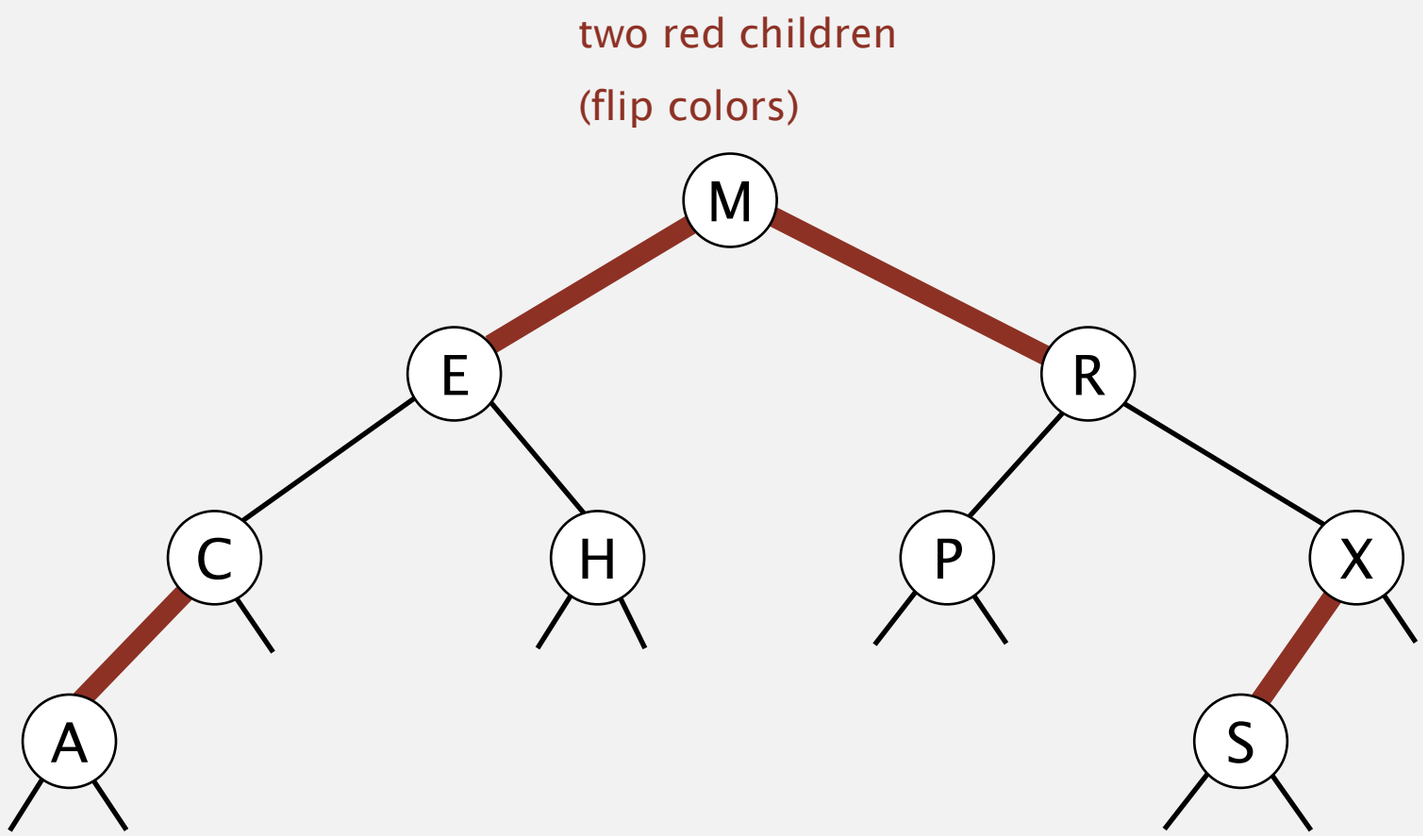
Red-black BST insertion



Red-black BST insertion

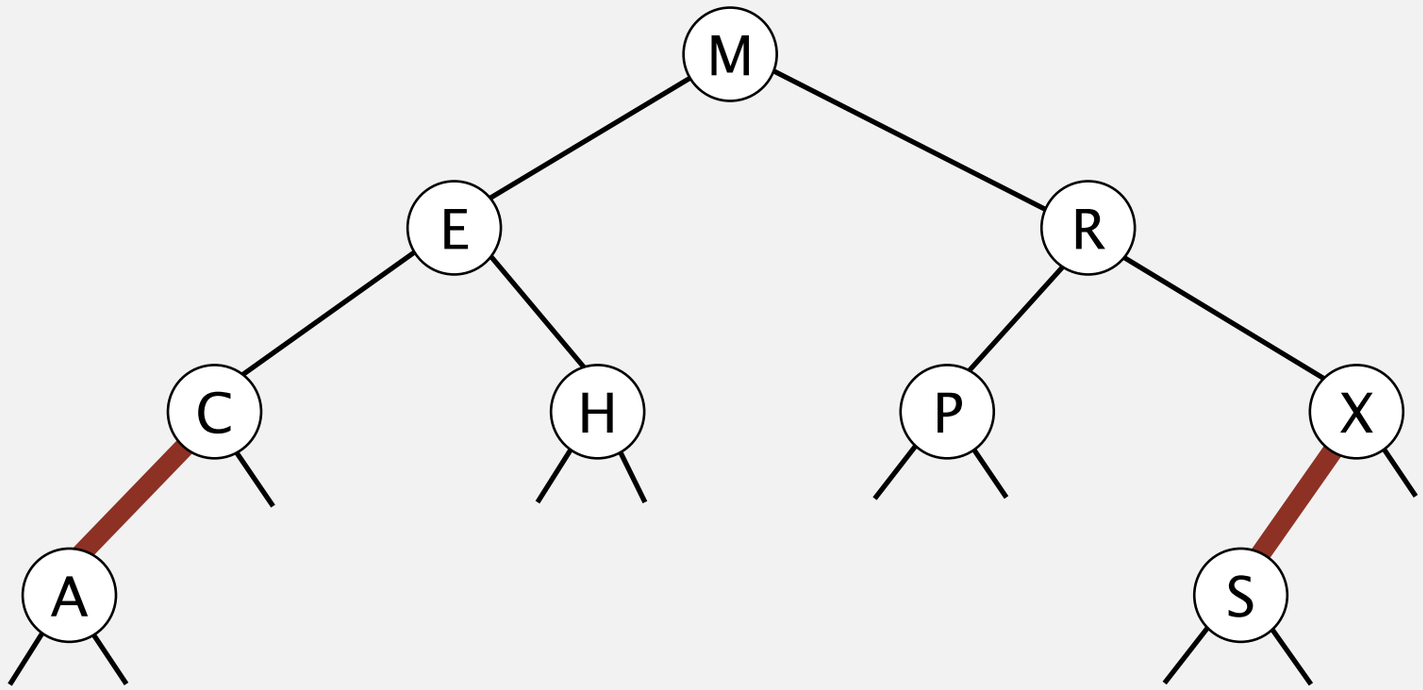


Red-black BST insertion



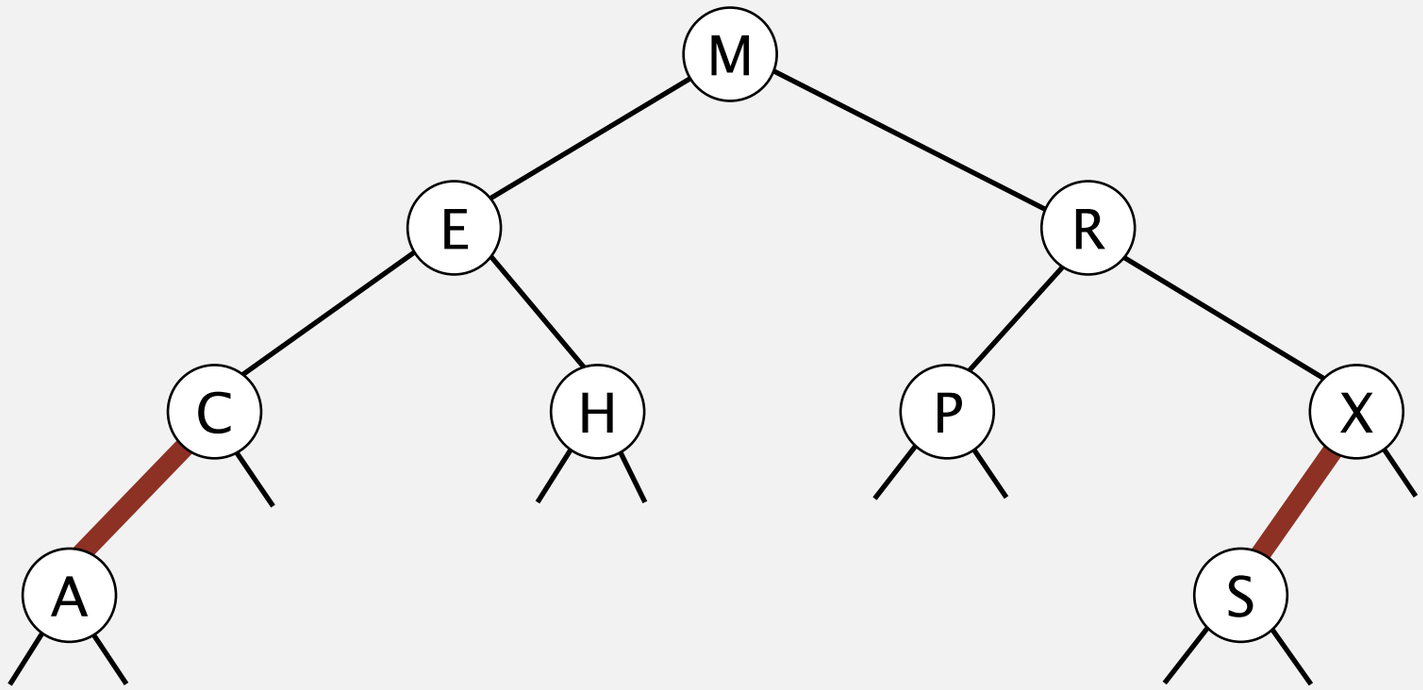
Red-black BST insertion

red-black BST



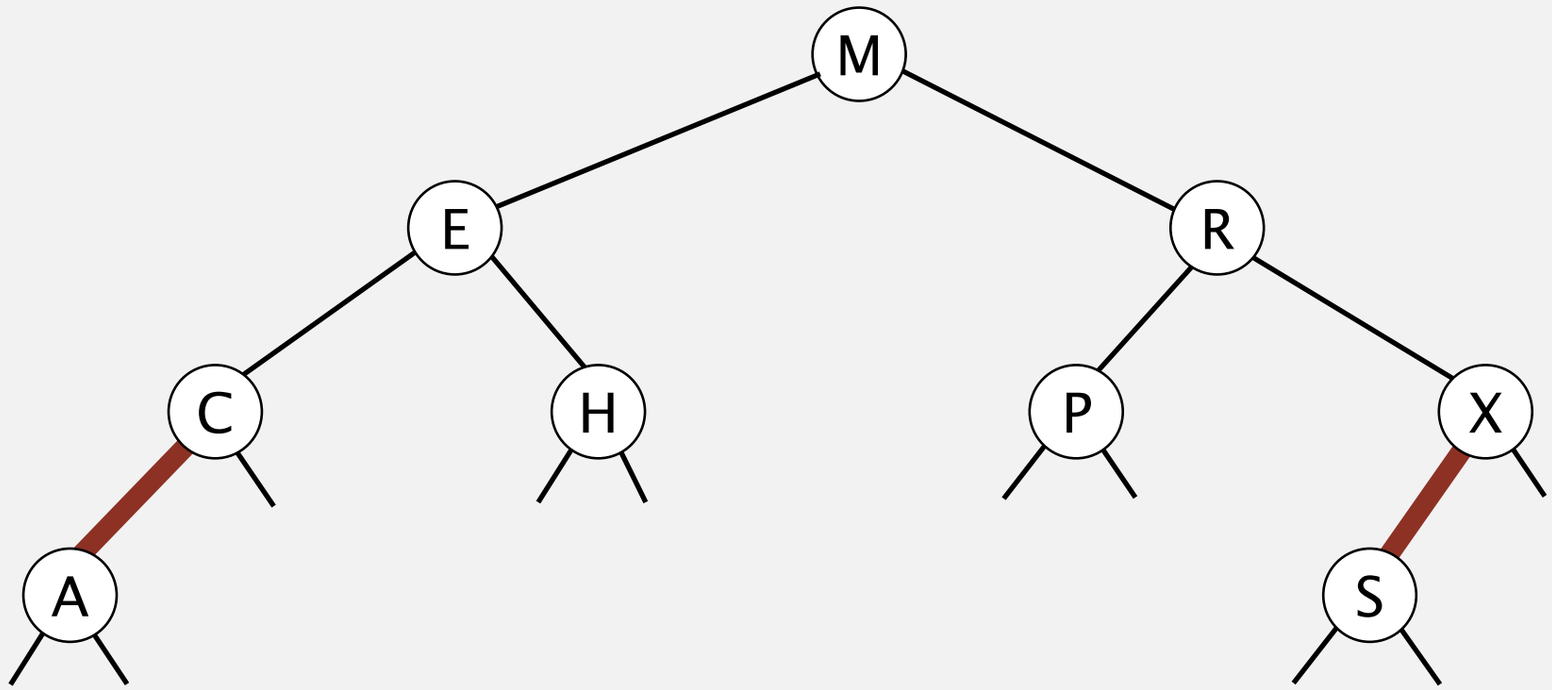
Red-black BST insertion

red-black BST



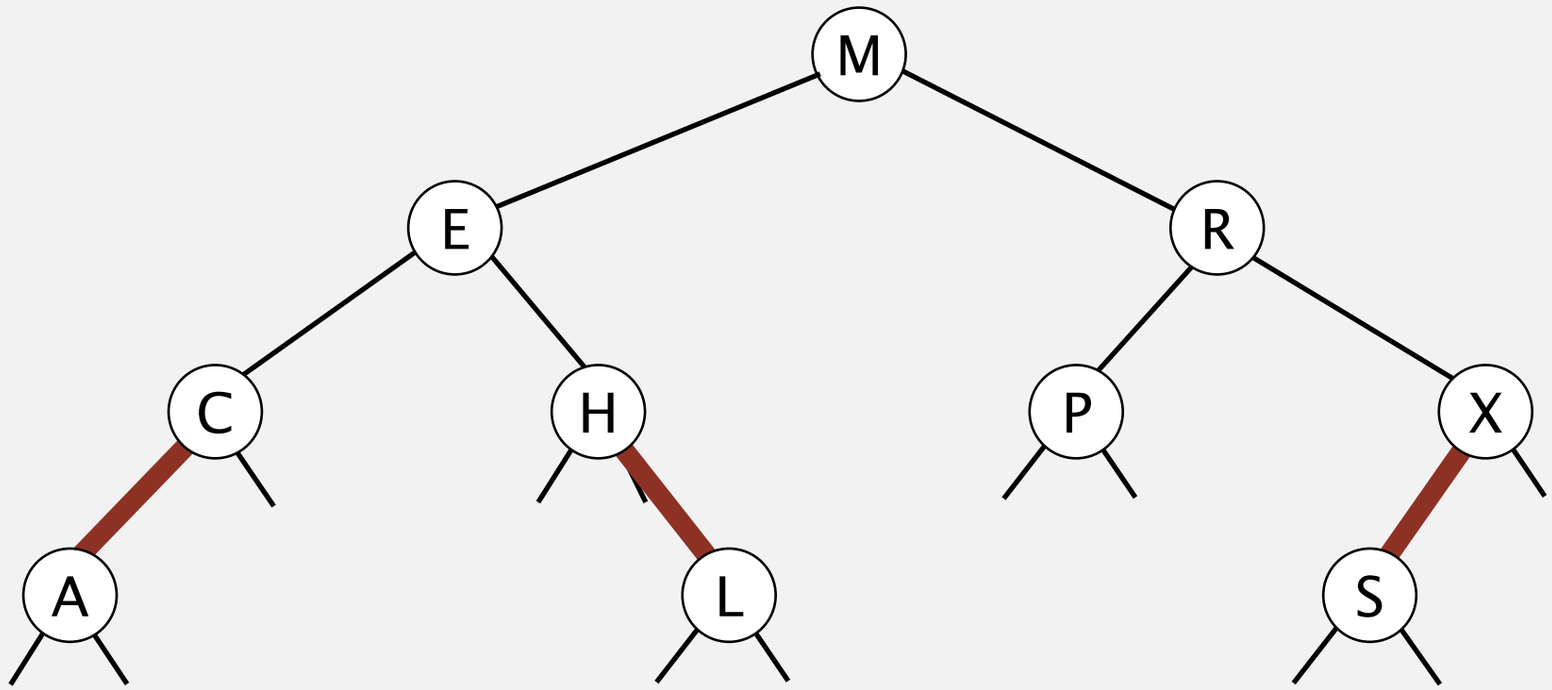
Red-black BST insertion

red-black BST



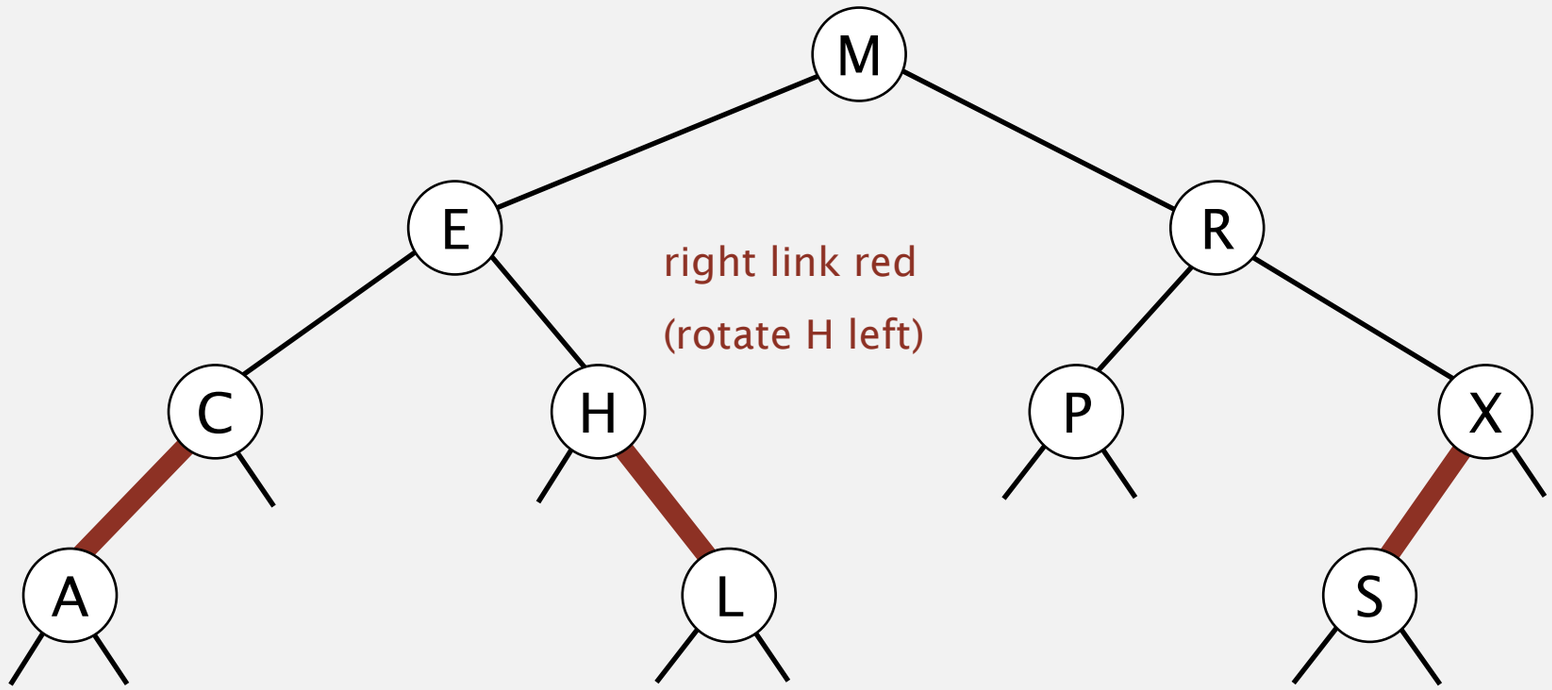
Red-black BST insertion

insert L



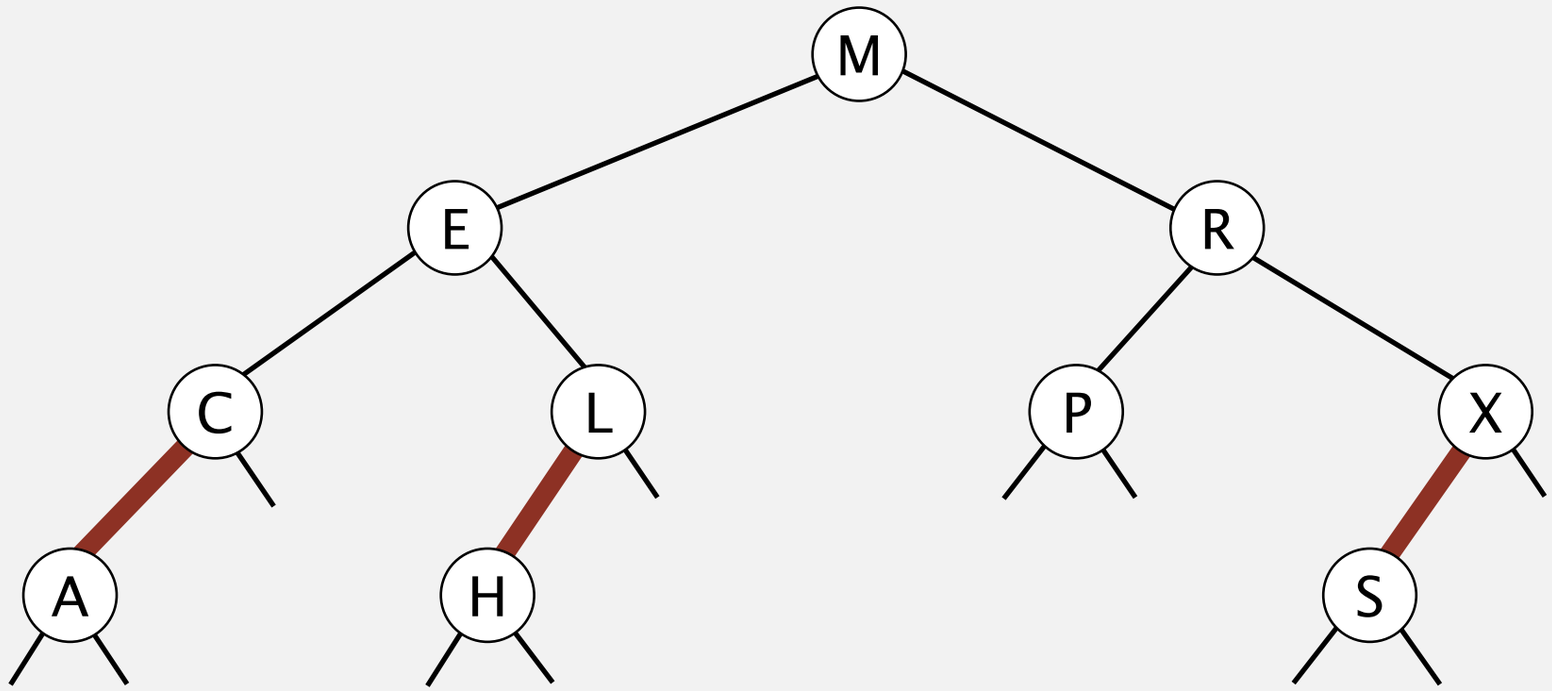
Red-black BST insertion

insert L



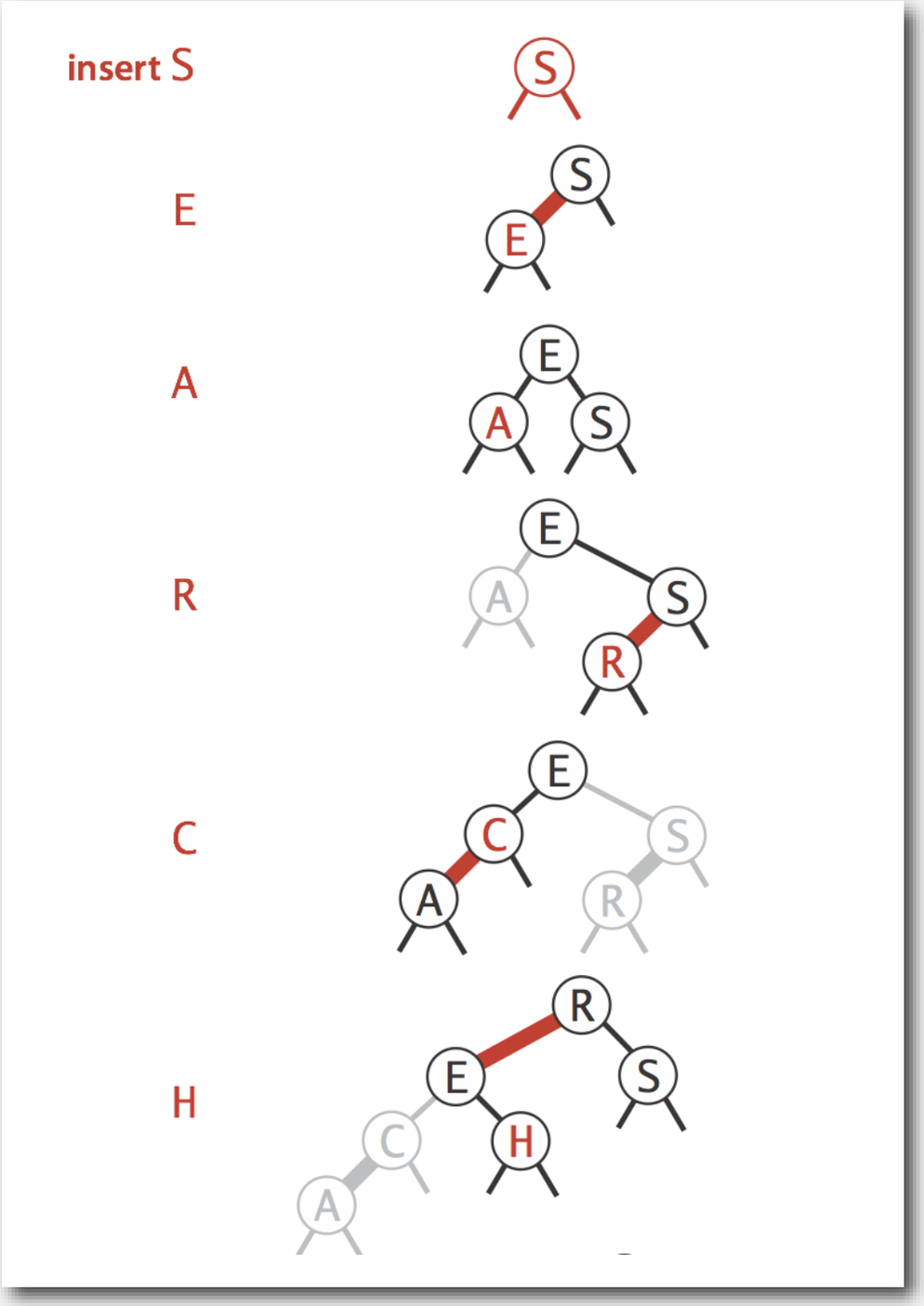
Red-black BST insertion

red-black BST



LLRB tree insertion trace

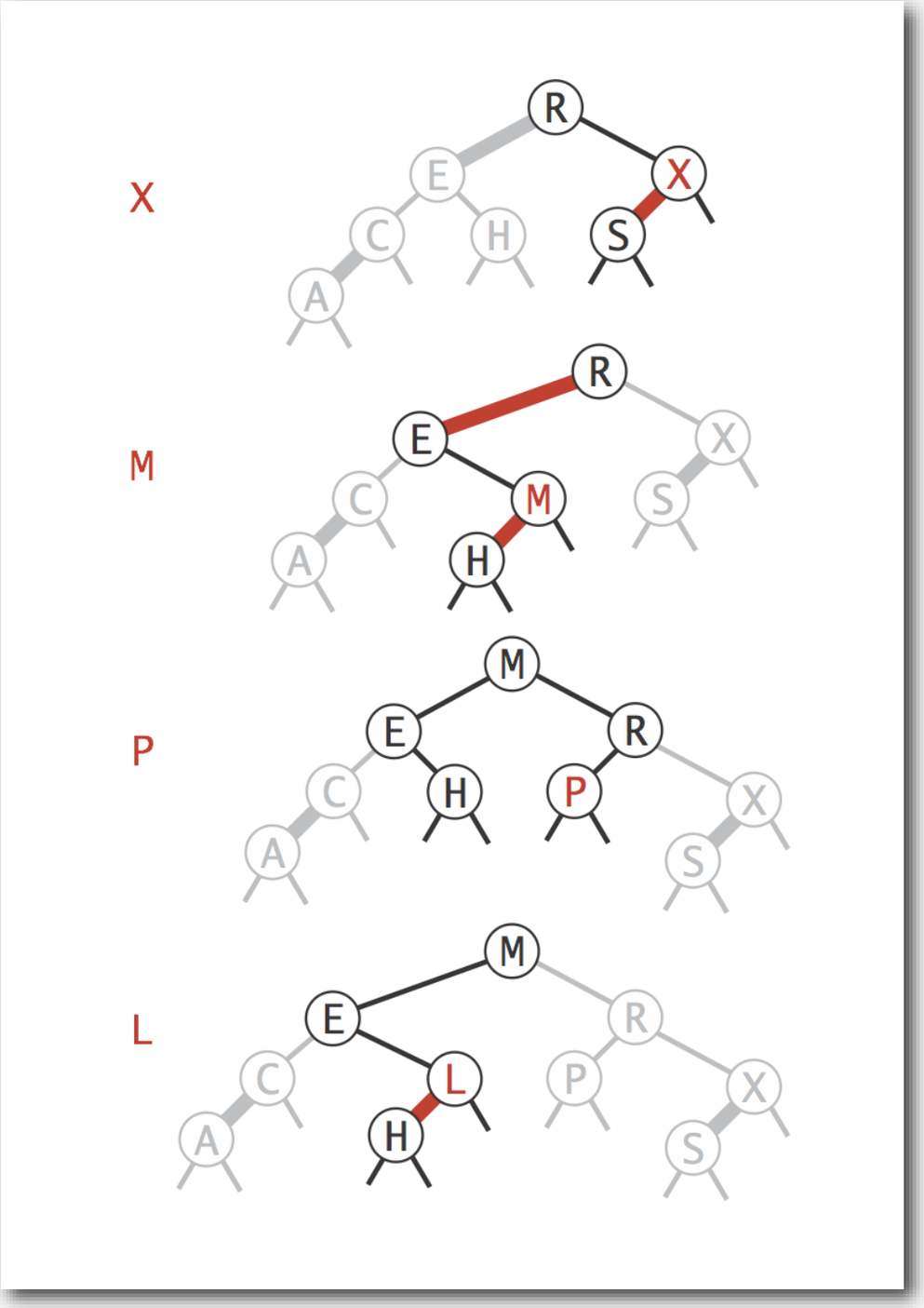
Standard indexing client.



red-black BST

LLRB tree insertion trace

Standard indexing client (continued).

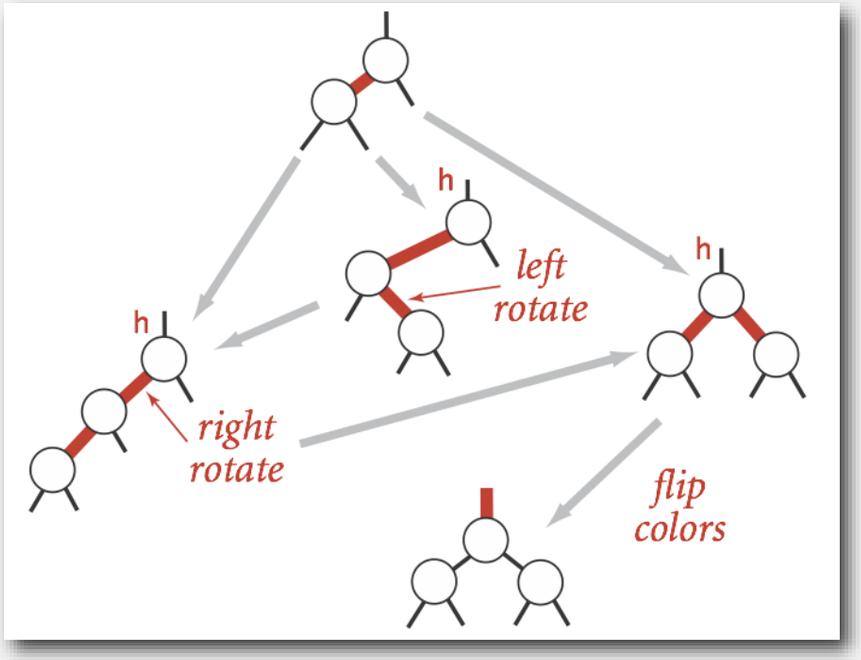


red-black BST

Insertion in a LLRB tree: Java implementation

Same code for both cases.

- Right child red, left child black: rotate left.
- Left child, left-left grandchild red: rotate right.
- Both children red: flip colors.




```
private Node put(Node h, Key key, Value val)
{
   if (h == null) return new Node(key, val, RED);
   int cmp = key.compareTo(h.key);
   if      (cmp  < 0) h.left  = put(h.left,  key, val);
   else if (cmp  > 0) h.right = put(h.right, key, val);
   else if (cmp == 0) h.val = val;

   if (isRed(h.right) && !isRed(h.left))     h = rotateLeft(h);
   if (isRed(h.left)  && isRed(h.left.left)) h = rotateRight(h);
   if (isRed(h.left)  && isRed(h.right))     flipColors(h);

   return h;
}
```

insert at bottom (and color red)

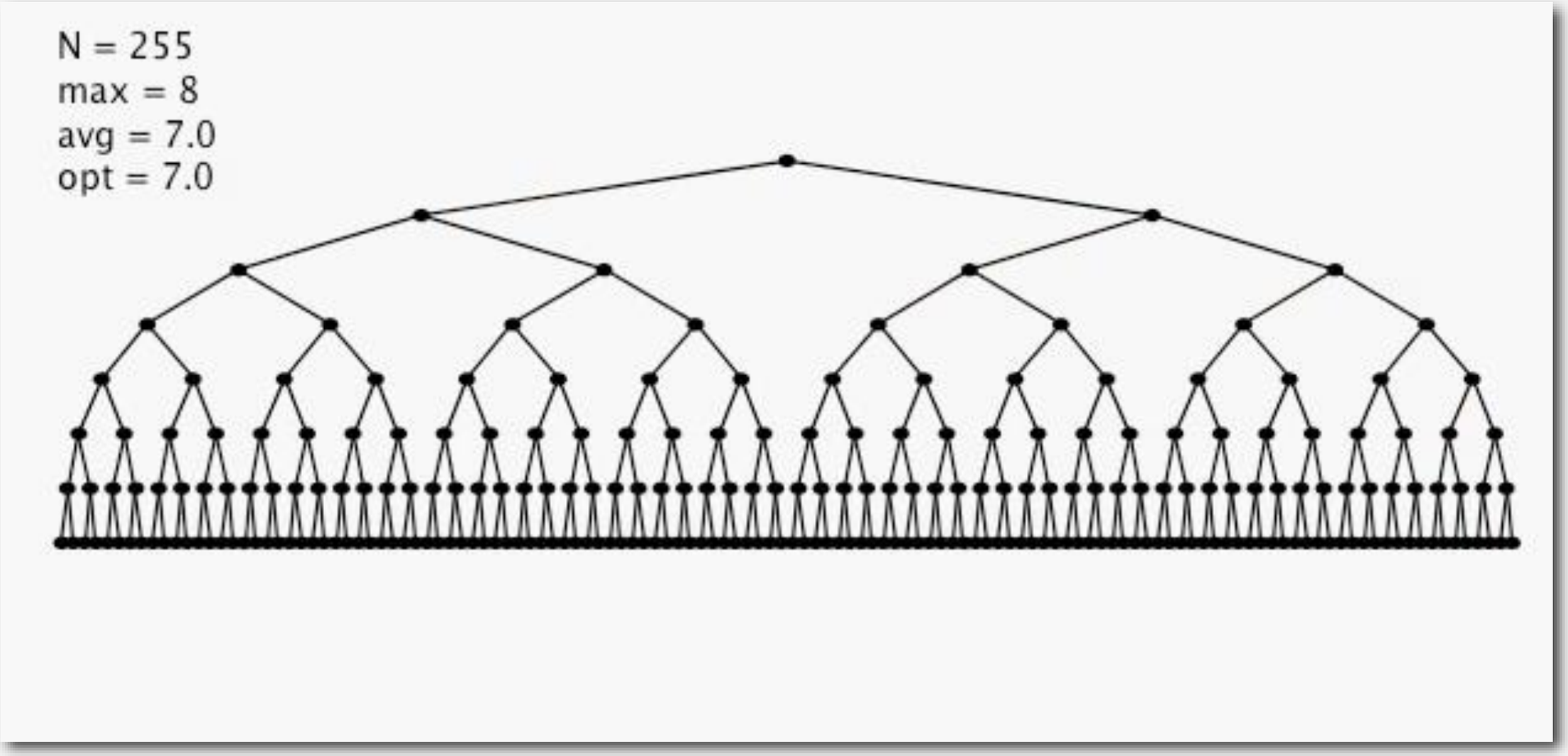
lean left

balance 4-node

split 4-node

only a few extra lines of code provides near-perfect balance

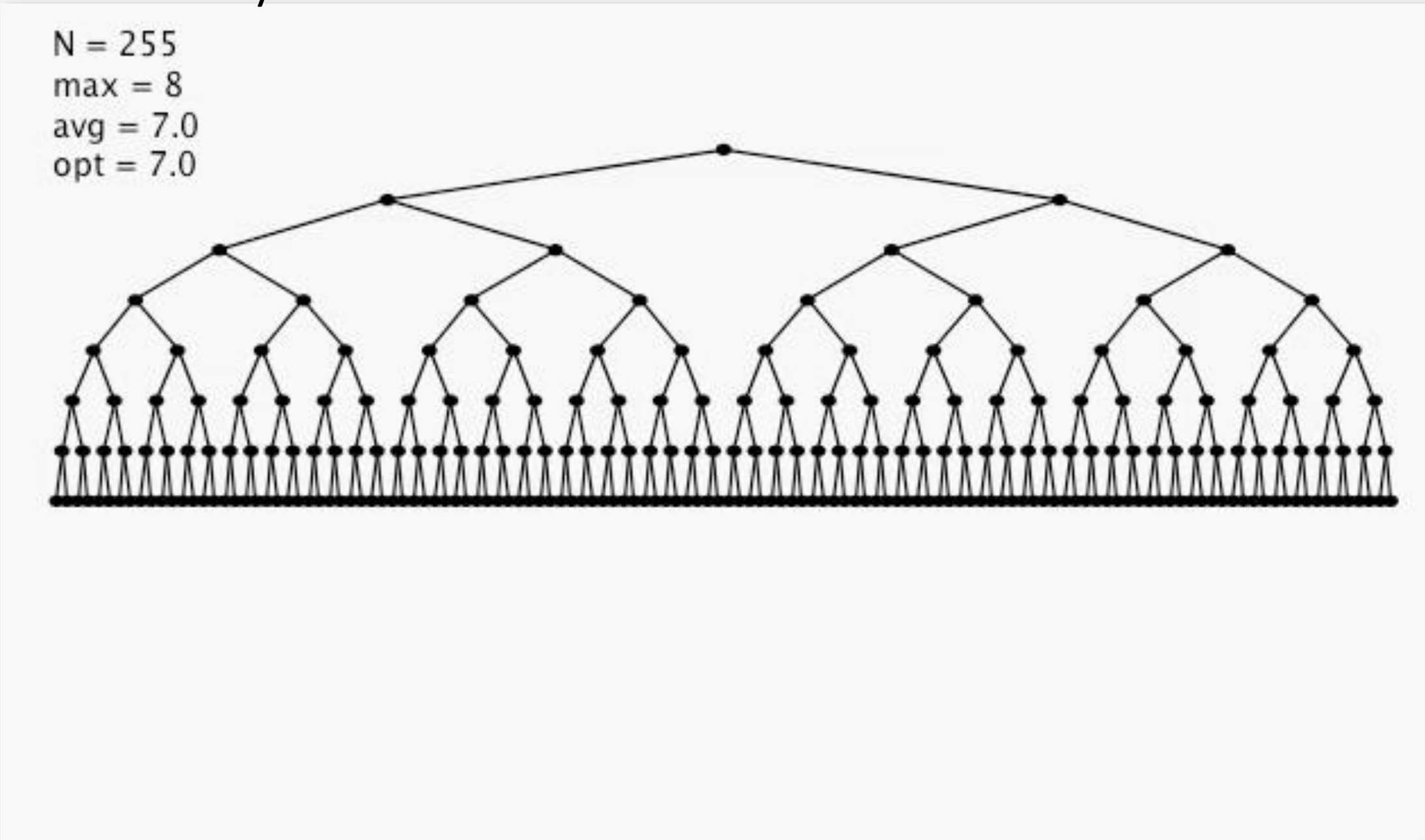
Insertion in a LLRB tree: visualization



255 insertions in ascending order

Insertion in a LLRB tree: visualization

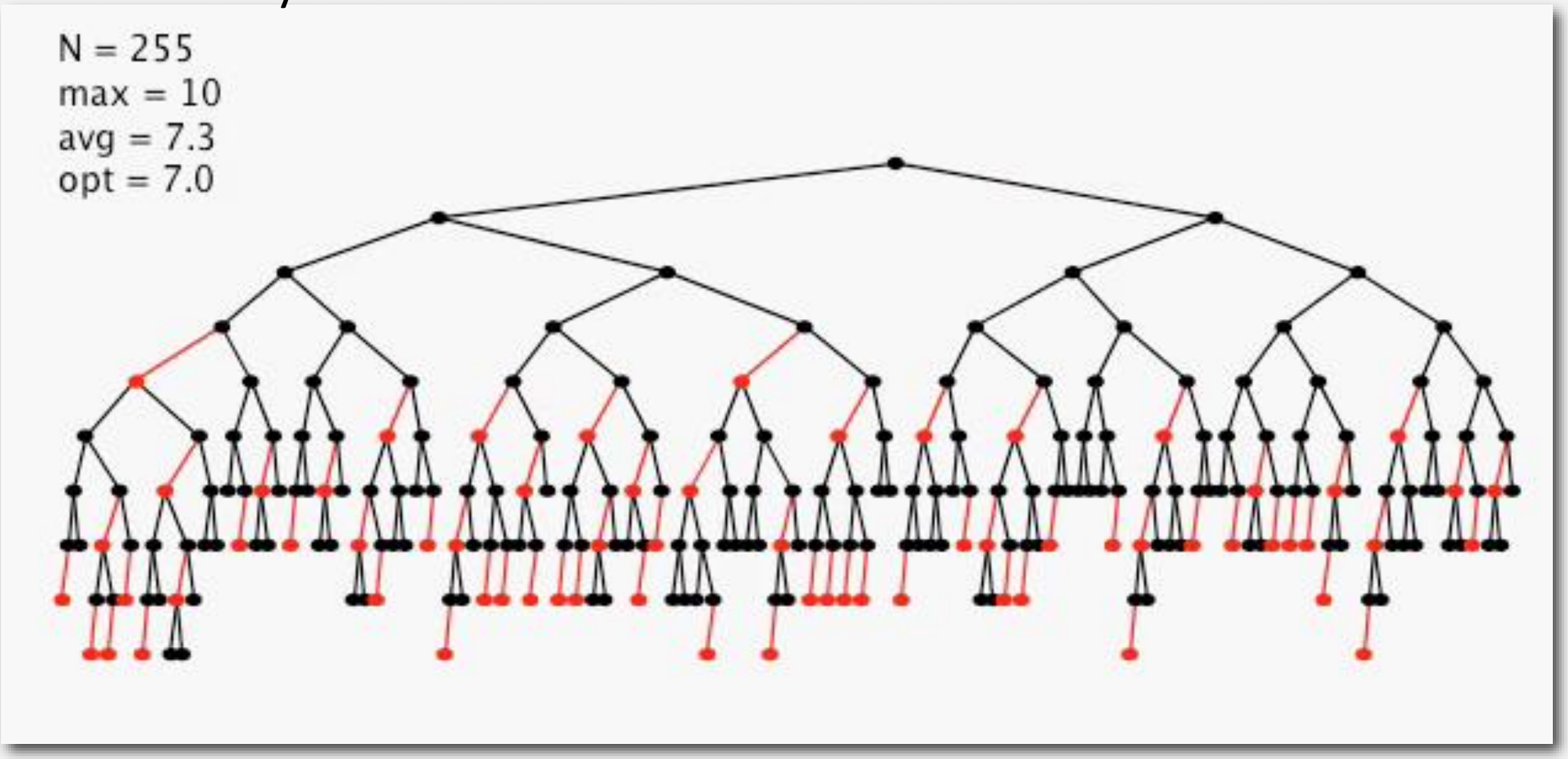
Remark. Only a few extra lines of code to standard BST insert.



255 insertions in descending order

Insertion in a LLRB tree: visualization

Remark. Only a few extra lines of code to standard BST insert.



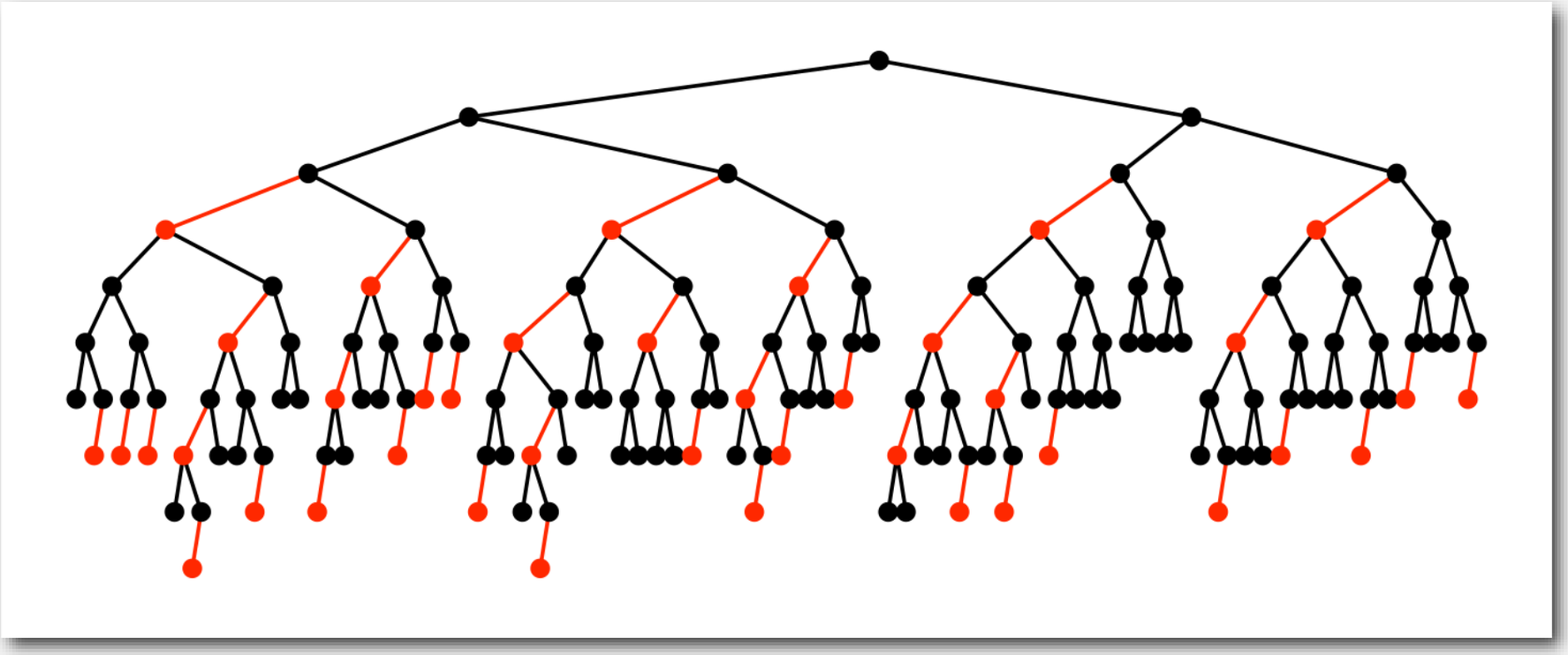
255 random insertions

Balance in LLRB trees

Proposition. Height of tree is $\leq 2 \lg N$ in the worst case.

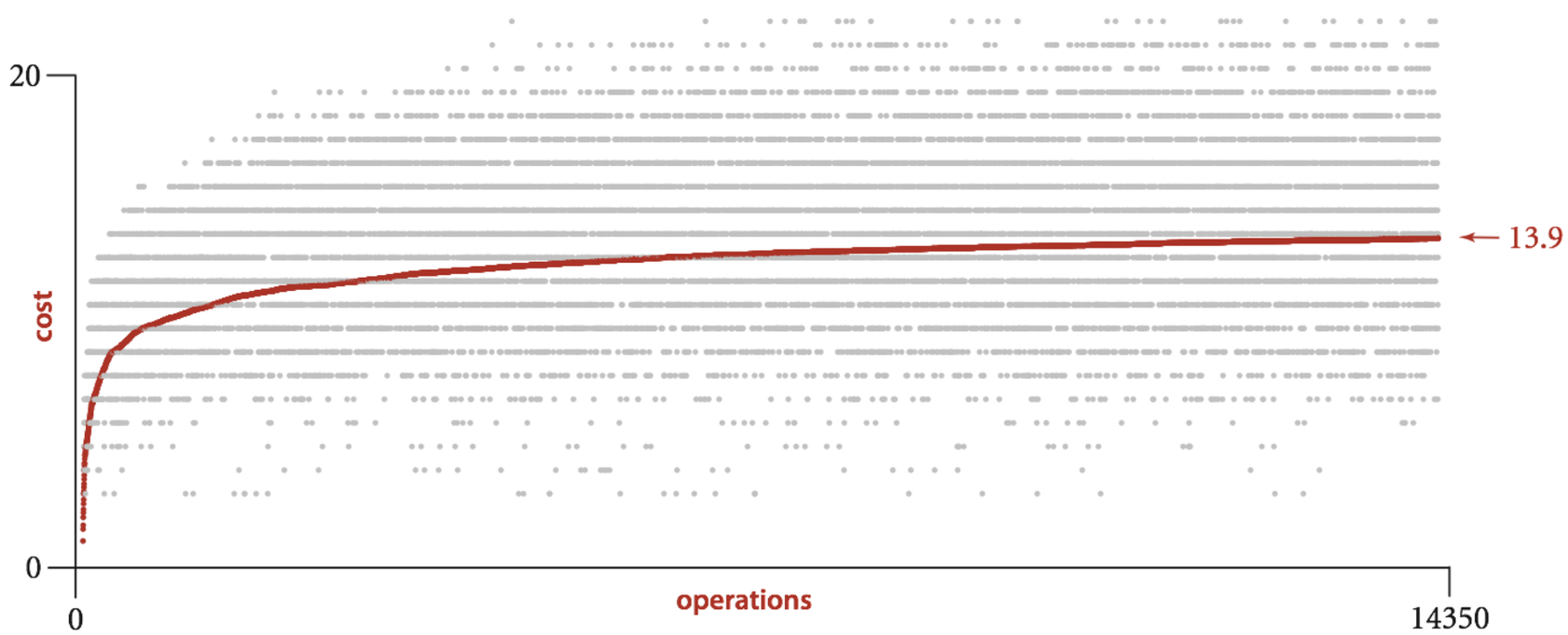
Pf.

- Every path from root to null link has same number of black links.
- Never two red links in-a-row.

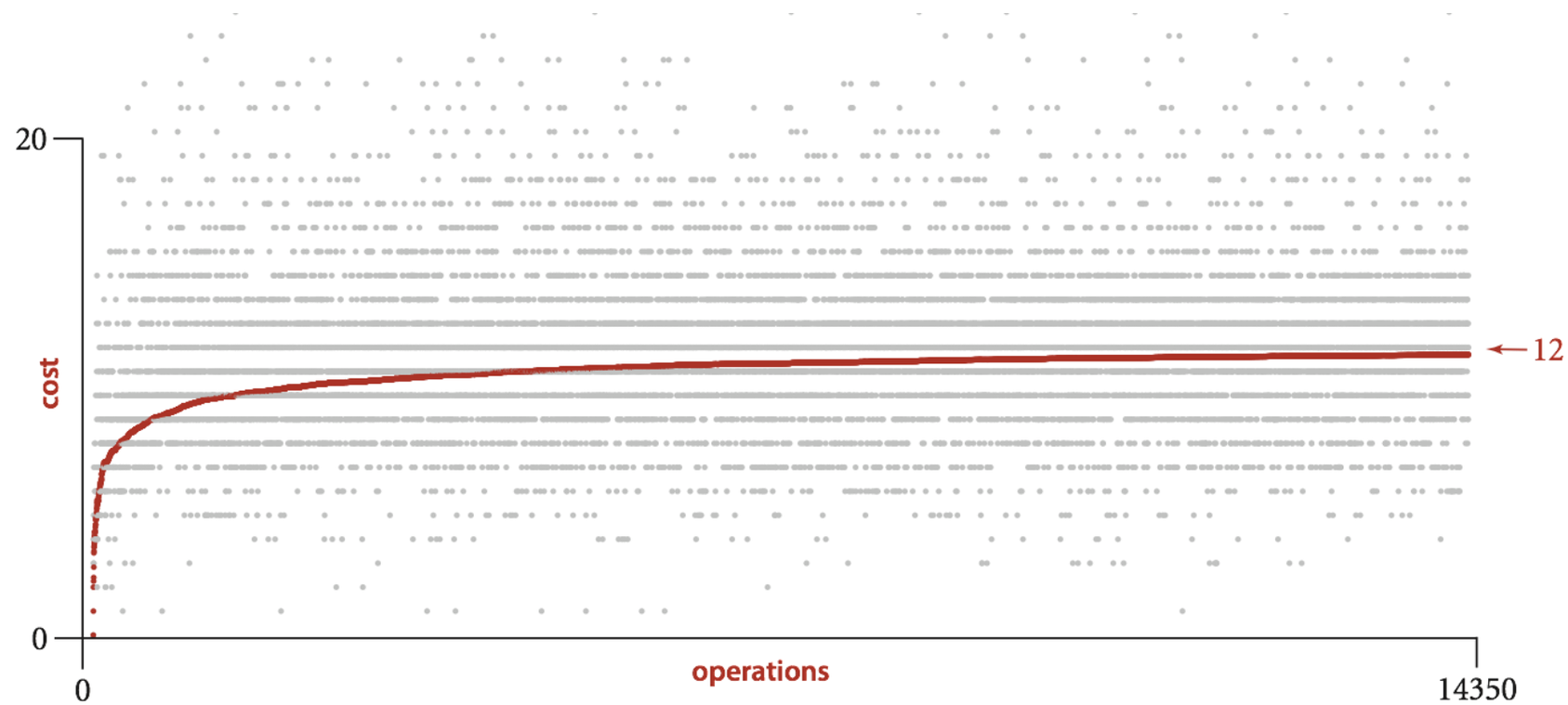


Property. Height of tree is $\sim 1.00 \lg N$ in typical applications.

ST implementations: frequency counter



Costs for `java FrequencyCounter 8 < tale.txt` **using** `BST`



Costs for `java FrequencyCounter 8 < tale.txt` **using** `RedBlackBST`

ST implementations: summary

implementation	worst-case cost (after N inserts)			average case (after N random inserts)			ordered iteration?
	search	insert	delete	search hit	insert	delete	
sequential search (unordered list)	N	N	N	N/2	N	N/2	no
binary search (ordered array)	lg N	N	N	lg N	N/2	N/2	yes
BST	N	N	N	1.39 lg N	1.39 lg N	?	yes
red-black BST	2 lg N	2 lg N	2 lg N	1.00 lg N *	1.00 lg N *	1.00 lg N *	yes

* exact value of coefficient unknown but extremely close to 1